

SIPNAM - Bitácora de Pasantía

Bitácora de Pasantía

SIPNAM — Sistema de Gestión Escolar

Nombre del proyecto: SIPNAM (Sistema de Información y Plataforma de Notificación de Asistencias y Novedades)

Tipo: Aplicación web (PWA) para gestión de asistencias, docentes, novedades, incidentes y supervisión escolar.

Descripción breve: Plataforma con roles diferenciados (director, vice, preceptor, secretario, conserje y supervisor) que permite registrar asistencias, gestionar novedades e incidentes, subir fotos y supervisar escuelas en tiempo real.

Stack tecnológico

Tecnología	Uso
React 19 + TypeScript	Framework de interfaz y tipado
Vite 8	Build y servidor de desarrollo
Firebase (Auth, Firestore, Hosting)	Autenticación, base de datos y despliegue
React Router v7	Navegación
react-hook-form + Zod	Formularios y validación
Recharts	Gráficos
jsPDF	Exportación de reportes PDF

Metodología

- Desarrollo guiado por **agentes de IA** (asistente de código).
- Cada tarea finalizada verifica: compilación (`tsc`), tests (`vitest`) y linting (`eslint`).
- Commits descriptivos con estilo convencional (`feat:` , `fix:` , `refactor:` , `perf:` , `docs:`).

Duraciones referenciales (resumen)

Fase	Tiempo
Instalación, configuración y documentación inicial (Semana 1)	~2 días
Total del proyecto	A definir por semana

Los tiempos son estimaciones del tiempo efectivo de trabajo en cada fase, detalladas semana a semana en esta bitácora.

Semana 1 — Instalación, Configuración y MVP

Período: miércoles 23 y jueves 24 de julio de 2026

Horas acumuladas: a completar por el pasante.

Objetivo de la semana

Poner en marcha el proyecto desde cero: instalar y configurar el stack, crear la documentación base, e implementar un **MVP funcional** con autenticación, tipos, servicios de datos y el panel del Supervisor.

Actividades realizadas

Miércoles 23 de julio

- **Instalación y configuración inicial del proyecto** (React + TypeScript + Vite).
 - Se instalaron todas las dependencias: react-router-dom, firebase, react-hook-form, zod, recharts, jspdf, lucide-react, motion, vitest, testing-library.
- **Documentación inicial del proyecto** (contexto, requerimientos, arquitectura, base de datos, API, diseño UI, setup Firebase, etc.).
- **Setup inicial completo** en el repositorio.

Jueves 24 de julio

- **Implementación del MVP:**
 - Tipos de datos, servicios y componentes comunes y formularios (commit `c0f267e`).
 - **Panel del Supervisor completo** con sub-vistas, tabs y rutas anidadas (commit `e0d0410`).
 - Corrección de bugs críticos y warnings del análisis de código.
 - **Rediseño del panel Supervisor:** listado de escuelas + vista por escuela.
 - Sección **Usuarios** en el detalle de escuela, con contador y meta de cada usuario.
 - **Formulario para crear escuelas** desde el panel del Supervisor + `addSchool` en firestore.
 - Resumen en el **home del Supervisor:** 3 cajas (asistencias, novedades, incidentes recientes), estadísticas del día, acciones rápidas y actividad reciente.
 - Links "Gestionar Escuelas" y "Configuración de Usuarios" en el menú del Supervisor.
 - Actualización del documento de tareas pendientes.
-

Dificultades encontradas

- Falta de un **índice compuesto** en Firestore (`orderBy` en `getSchools`). Se resolvió quitando el `orderBy` y ordenando en el cliente.
 - Import CSS roto en `SupervisorSchoolDetail` tras reorganizar componentes; se corrigió el import.
-

Resultados y evidencias

- Proyecto corriendo localmente en `http://localhost:5173`.
 - Panel de Supervisor funcional: listado de escuelas, detalle por escuela, usuarios y creación de escuelas.
 - 16 commits realizados en la semana (instalación, documentación y MVP).
-

Aprendizajes

- Configuración de Firebase (Auth + Firestore) y reglas de acceso por rol.
 - Uso de React Router v7 con rutas protegidas y anidadas.
 - Diseño de componentes reutilizables y organización por carpetas.
-

Pendientes / Próxima semana

- Verificación de asistencias, historial y reportes.
- Módulos de asistencia de gestión y de docentes.
- Campos ampliados en novedades/incidentes y fotos.

Semana 2 — Asistencias, Historial y Reportes

Período: martes 11 de agosto de 2026

Horas acumuladas: a completar por el pasante.

Objetivo de la semana

Expandir el sistema con los módulos de **asistencia de gestión** y **asistencia de docentes**, agregar verificación de asistencias, historial, reportes CSV, fotos y endurecer las reglas de Firestore.

Actividades realizadas

- **Asistencia de docentes, foto diaria y rediseño de la asistencia de gestión** (commit `9e62815`).
 - Nuevo flujo para cargar asistencia de docentes con **foto diaria**.
 - Rediseño del registro de asistencia de gestión.
 - **Campos ampliados en novedades/incidentes y fotos en base64** (commit `b52b497`).
 - Se ampliaron los campos de novedades e incidentes.
 - Las **fotos se guardan en base64 comprimido directamente en Firestore** (sin Firebase Storage), para simplificar el almacenamiento.
 - **Verificación de asistencias, historial, reportes CSV y reglas Firestore endurecidas** (commit `e3b0f35`).
 - Módulo de verificación y consulta de asistencias.
 - **Historial** con exportación de **reportes CSV**.
 - Reglas de seguridad de Firestore reforzadas para limitar el acceso por rol y `escuelaId`.
-

Dificultades encontradas

- Manejo del **tamaño de las fotos** en Firestore almacenadas en base64 (se resolvió con compresión previa).
 - Definición de reglas de Firestore que no rompieran las consultas filtradas por escuela (ajuste de reglas y mantenimiento de índices).
-

Resultados y evidencias

- Asistencia de docentes con foto diaria funcional.
 - Novedades e incidentes con campos ampliados y fotos.
 - Historial consultable y exportable a CSV.
 - 3 commits en la semana.
-

Aprendizajes

- Almacenamiento de imágenes en Firestore como base64 comprimido.
 - Escritura de reglas de seguridad de Firestore orientadas a roles.
 - Exportación de datos a CSV para reportes.
-

Pendientes / Próxima semana

- Implementar **tiempo real** (onSnapshot) en los paneles.
- Tema oscuro / apariencia.
- Home para directores y preceptores.
- Filtros, paginación y optimizaciones de rendimiento.

Semana 3 — Tiempo Real, Home, Tema y Gestión de Usuarios

Período: lunes 17 y martes 18 de agosto de 2026

Horas acumuladas: a completar por el pasante.

Objetivo de la semana

Llevar el sistema al siguiente nivel: **datos en tiempo real** con `onSnapshot`, página de inicio (Home) para directores/preceptores, tema oscuro, filtros, paginación, optimización de rendimiento y gestión completa de usuarios y escuelas desde el panel del Supervisor.

Actividades realizadas

Lunes 17 de agosto

- **Tiempo real con `onSnapshot` en todos los paneles** (commit `fd1c364`): las asistencias, novedades e incidentes se actualizan al instante.
- **Home para directores/preceptores** con info de la escuela y actividad del día.
 - Uso de queries filtradas por `escuelaId`.
 - `SchoolSelect` pasa a usar `getSchoolById` para directores/preceptores.
 - Se elimina `SchoolSelect` de los formularios y se usa la escuela del perfil.
- **Página de configuración de apariencia con tema oscuro.**
- **Mejora estética e información de las secciones de escuelas.**
- **Vista Hoy/Histórico** en el detalle de escuela y grid de 2 columnas.
- **Editar usuarios y restablecer contraseña** desde el Supervisor.
- **Tooltips CSS y labels** para botones de usuario.
- **Filtros de tipo y categoría en Historial y paginación client-side.**
- **Optimizaciones:** `React.memo` en 10 componentes de lista y `Suspense fallback` para lazy loading de rutas.
- **Correcciones:** botón de volver en el Panel de Supervisión, `HydrateFallback` de `react-router v7`, `enableIndexedDbPersistence` deprecado reemplazado por `initializeFirestore`, mapeo de errores Firebase a mensajes amigables en Login, sincronización del Home al navegar.
- **Refactor:** migrar form de escuela a `react-hook-form` + `Zod`, descomponer `SupervisorSchoolDetail` en subcomponentes, consolidar estados con `hook useFeedback`, eliminar duplicación de `dateKey`.
- **Nuevo:** `ErrorBoundary` global, confirmación antes de eliminar fotos/desactivar usuarios.
- **Tests de componentes** (Login, Novedades, Incidentes, `AttendanceRow`, `StatusBadge`, `NotFound`).
- **Documentación:** creación de `CONTEXT.md` y actualización de docs para retomar el proyecto.

Martes 18 de agosto

- **Gestión de escuelas/docentes desde Supervisor:** editar/eliminar escuelas y editar docentes.
- **Simplificación de asistencia de docentes a subida de foto.**
- **Optimización del Home:** reemplazo de suscripciones real-time por `fetch` periódico para refrescar actividad.
- **Correcciones:** bugs de `StrictMode`, consistencia de `dateKey`, error handling,

condición de carrera en el login.

- **UI/UX** (ver Semana 4 para el detalle de animaciones, o agrupado aquí):
 - Skeleton loading placeholders y unificación de design tokens CSS.
 - Dark mode accent colors y "empty states" amigables.
 - Animaciones de entrada, botón con spinner/success.
 - Breadcrumbs, contadores animados y barra de filtros animada.
 - Timeline visual y focus-visible rings + backdrop blur en modales (accesibilidad).
 - **Actualización de** `CONTEXT.md` y tareas pendientes.
-

Dificultades encontradas

- **Condición de carrera en el login:** el perfil podía no estar cargado antes de resolver el inicio de sesión. Se resolvió esperando la carga del perfil.
 - **Re-render infinito** al usar suscripciones en tiempo real; se corrigió y se optó por fetch periódico en el Home.
 - **Error de Firestore** `undefined` y **bug de zona horaria (timezone)** en las fechas.
 - `enableIndexedDbPersistence` **deprecado:** migración a `initializeFirestore`.
 - **Índices compuestos faltantes** en Firestore para queries con `orderBy`.
 - **Nombres de archivos (dateKey) inconsistentes** entre el detalle de escuela y la asistencia.
-

Resultados y evidencias

- Datos en tiempo real en todos los paneles de supervisión.
 - Home informativo para directores/preceptores con actividad del día.
 - Tema oscuro configurable en `/tema`.
 - Gestión de usuarios, escuelas y docentes desde el Supervisor.
 - Filtros, paginación y optimizaciones de rendimiento.
 - Suite de tests de componentes funcionando.
-

Aprendizajes

- Uso de `onSnapshot` para suscripciones en tiempo real y sus consideraciones de rendimiento.
 - Optimización de renders con `React.memo` y lazy loading.
 - Migración de APIs deprecadas de Firestore.
 - React Hook Form + Zod para formularios robustos.
-

Pendientes / Próxima semana

- Componentes de UI más pulidos (animaciones, notificaciones, offline).
- Lógica de notificaciones en tiempo real para supervisores.
- SVG, impresión, PWA y ajustes de diseño responsive.

Semana 4 — PWA, Notificaciones y Pulido de la Interfaz

Período: miércoles 19, viernes 21 y sábado 22 de agosto de 2026

Horas acumuladas: a completar por el pasante.

Objetivo de la semana

Convertir la app en una **PWA instalable** con navegación móvil, agregar notificaciones en tiempo real, búsqueda global, exportación PDF, gráficos y **pulir toda la interfaz** con animaciones y accesibilidad.

Actividades realizadas

Miércoles 19 de agosto

- **Soporte PWA + barra de navegación inferior para móviles** (commit 51cbf28).
 - La app es instalable desde el navegador.
 - Botón de menú (hamburguesa) visible en todas las resoluciones (fix de navbar para tablet 768–1024px).
 - Botón de volver con mayor área táctil, separado en modo icono/texto.
- **Notificaciones en la app con actualización en tiempo real** (commit 292ad43): campana de notificaciones.
- **Búsqueda global con atajo Ctrl+K** (commit 88922ad).
- **Exportación de Historial a PDF** con jsPDF + autoTable.
- **Gráficos de dashboard con Recharts** en el home del Supervisor.
- **Sistema de toasts animados** (commit f41460c).
- **Pantalla de carga (splash) animada** con logo, barra de progreso y gradiente.

Viernes 21 de agosto

- **Retención y trazabilidad de incidentes:**
 - Registro del **historial de cambios de estado de incidentes** en Firestore.
 - Visualización del historial de estado a supervisor y escuelas.
 - **Queries de alcance jurisdiccional** y exportación CSV de respaldo a nivel de jurisdicción.
 - Tarjeta de respaldo de datos con exportación CSV.
 - **Banner de advertencia de purga de datos a fin de año** para el supervisor.
- **Indicadores en tiempo real** en todas las vistas del supervisor (commit e04683c).
- **Animaciones con motion y auto-animate :**
 - Listas de actividad en vivo (auto-animate).
 - Inserción/eliminación en el grid de escuelas.
 - Secciones de acordeón expansibles/contraíbles.
 - Entrada/salida de toasts.
 - Eventos del historial de estado de incidentes.
 - **View transitions** en la navegación entre páginas.
 - **Morph** de la tarjeta de escuela hacia la página de detalle.
 - Confirm dialog con entrada/salida animada, banner de retención, lightbox con zoom.
- **Normalización de formato** (endOfLine auto) y corrección de bloqueos de build

TS.

Sábado 22 de agosto

- **Paleta de colores derivada, acentos con gradiente y aurora animada** (commit `29af763`).
 - **Crossfade de skeleton a contenido** en pantallas principales.
 - **Indicador deslizante activo en la barra de navegación inferior.**
 - **Acciones de tarjetas de escuela movidas a la fila inferior.**
 - **Actualización de documentación** (CONTEXT) con animaciones, tiempo real y sesión de color.
-

Dificultades encontradas

- Colisión de nombres de tipo `School` con `lucide-react` (se renombró el alias del import).
 - Mantener las animaciones fluidas sin afectar el rendimiento en listas grandes (se usó `auto-animate` con cuidado).
 - Configurar PWA y vista previa de la build de producción para validar la instalación móvil.
-

Resultados y evidencias

- Aplicación instalable (PWA) con barra de navegación inferior en móvil.
 - Notificaciones en tiempo real y búsqueda global (Ctrl+K).
 - Exportación de Historial a PDF y backup CSV jurisdiccional.
 - Gráficos Recharts e interfaz animada (toasts, splash, view transitions, aurora).
 - Trazabilidad del historial de cambios de estado de incidentes.
-

Aprendizajes

- Implementación de PWA (manifest, service worker) con Vite PWA plugin.
 - Uso de `motion` y `auto-animate` para animaciones declarativas.
 - Uso de **View Transitions API** para transiciones de navegación.
 - Exportación PDF con jsPDF + autoTable y CSV a nivel jurisdicción.
-

Pendientes / Próxima semana

- Diseño responsive de la pantalla de **login** (split-screen escritorio, full-screen móvil).
- Ajustes finales de estilo y correcciones de build.
- Cierre de sesión por inactividad para el rol director.
- Changelog, mapa de calor y animaciones finales.

Semana 5 — Login Responsive, Onboarding y Cierre de Sesión por Inactividad

Período: lunes 24 al jueves 27 de agosto de 2026

Horas acumuladas: a completar por el pasante.

Objetivo de la semana

Finalizar la experiencia de usuario: **login responsive** (split-screen en escritorio, full-screen en móvil), centro de ayuda y onboarding, notificaciones y feedback offline, indicadores de actividad, y **cierre de sesión por inactividad** para el rol director.

Actividades realizadas

Lunes 24 de agosto

- **Auditoría y permisos:** trail de auditoría para usuarios y docentes gestionados por el supervisor, protección de acceso a perfiles en reglas Firestore (`exists()`), suscripciones de notificaciones limitadas a la escuela del usuario, carga del perfil vía `onAuthStateChanged`.
- **UX:**
 - Fotos de incidentes, validación de rango de fechas, estado activo del drawer, toggle de contraseña.
 - Feedback de sincronización offline para incidentes.
 - Feedback offline para asistencias y novedades.
 - **Validación de tipo y tamaño de imagen** antes de subir a Firestore.
- **Notificaciones:** alertas de subida en vivo para supervisor con notificaciones nativas (opt-in).
- **Onboarding / Ayuda:**
 - Centro de ayuda con FAQ según rol, guía offline, pasos de instalación y glosario.
 - Welcome tour según rol en el primer login.
 - Prompt de instalación PWA inteligente con instrucciones para iOS.
 - Hints contextuales descartables en formularios clave.
 - Empty states con botones de acción.
- **Otras:** acceso directo PWA, avisos de feriados argentinos y recordatorio de asistencia, animaciones ambientales solo con conexión Wi-Fi, SEO con enfoque en Tinogasta y búsqueda mejorada, suite de tests global.
- **Actualización de documentación** (permissions, audit, UX).

Martes 25 de agosto

- **Extras de interfaz:**
 - Página de perfil de usuario (`/perfil`).
 - Skeletons para Fotos y mejora del de Historial.
 - Pull-to-refresh en Home e Historial.
 - Banner de estado rápido con resumen de actividad del día en Home.
 - Badges en la barra de navegación inferior (incidentes abiertos y asistencia pendiente).
 - Feedback háptico en envíos de formularios (hook `useHaptic`).
 - Transiciones de página mejoradas.

- **Toggle de tema oscuro en el drawer**, utilidades de tema compartidas y corrección del arranque del tema oscuro.
- **Sparklines** con tendencia de 7 días en las estadísticas del supervisor.
- **Modal de changelog** mostrando funciones nuevas en el primer acceso tras actualizaciones.
- **Mapa de calor de asistencia** (30 días) en el home del Supervisor.
- **Confetti** en el envío del formulario de Novedades.
- **Gestos de swipe** con `SwipeableRow` en las tarjetas de incidentes del Historial.
- Indicador de cola offline con contador y animación.
- **Toasts de notificación en tiempo real** para supervisores (nuevos incidentes, asistencias, novedades).
- **Auto-guardado de campos de formulario** con hook `useFormDraft`.
- Ajustes para **impresión / salida PDF**.
- **Correcciones:** `netlify.toml` con SPA redirects y cache headers, WelcomeTour con CSS puro, errores de Firestore amigables, atajos de teclado (Ctrl+K y Escape).

Miércoles 26 de agosto

- **Login responsive:**
 - **Split-screen** con sidebar de marca para escritorio (commit `ac171e7`).
 - **Full-screen para móvil/tablet** con gradiente sutil.
 - Corrección de imports/variables sin uso para arreglar el build (TS6133).

Jueves 27 de agosto

- **Pulido del login:**
 - Login blanco móvil/tablet con círculos decorativos en esquinas.
 - Home/dashboard fluido y responsive en PC.
 - Eliminación de capas de fondo duplicadas y aurora ambiente solo en celulares.
 - Re-aplicación del login auto-suficiente para evitar doble inicio de sesión.
 - Mantener habilitada la opción de estado de incidente actual para evitar el select congelado.
- **Cierre de sesión por inactividad (rol director):**
 - Se implementó en `src/context/AuthContext.tsx`: tras **10 minutos sin actividad** (mouse, teclado, touch, scroll, clic) se cierra la sesión automáticamente.
 - **Solo aplica al rol director**; el **supervisor** mantiene la sesión abierta (pensado para permanecer logueado en el panel de supervisión).

Dificultades encontradas

- Evitar el **doble inicio de sesión** (login no auto-suficiente): se re-aplicó la lógica que espera la carga del perfil antes de resolver.
- Capas de fondo duplicadas y aurora visible en PC; se restringió a móvil y se limpiaron los orbs duplicados.
- WelcomeTour no se mostraba por conflicto con `motion`; se reescribió con CSS puro.
- Mantener el **estado activo del incidente** seleccionable (select se congelaba al deshabilitar la opción actual).

Resultados y evidencias

- Login responsive y pulido en todas las resoluciones.
- Centro de ayuda, onboarding y PWA install prompt.
- Feedback offline completo (asistencias, novedades, incidentes) con indicador de cola.

- Mapa de calor, sparklines, changelog, confetti, swipe y auto-guardado.
 - **Cierre de sesión por inactividad de 10 minutos para el rol director; sesión del supervisor sin expirar.**
-

Aprendizajes

- Diseño responsive de login con split-screen y variantes móvil/tablet.
 - Implementación de un **idle timer** con listeners de eventos del navegador y cleanup.
 - Personalización de la expiración de sesión por rol.
 - Feedback háptico, swipe gestures y auto-guardado de formularios.
-

Pendientes / Próxima semana

- Pruebas finales en distintos dispositivos.
- Validación de reglas de Firestore en producción.
- Preparación de la entrega y documentación final.

00 - Instalación y Configuración Inicial del Proyecto

Semana 1 · 23 al 24 de julio de 2026

Este es el primer documento de la bitácora. Documenta **cómo se instaló y configuró** el proyecto desde cero, el **porqué** de cada decisión, y la **documentación inicial** que se hizo para entender de qué se trataba el proyecto y qué había que construir.

Cuando este documento se convierta a Word, cada semana posterior detallará las actividades específicas realizadas en esa semana. Esta semana se completó en **2 días de trabajo**.

1. Por qué este stack

React + TypeScript con Vite

- **Por qué React:** ecosistema maduro, componentes reutilizables (ideal para las pantallas repetitivas de asistencias/escuelas) y gran soporte para el tipo de app (formularios, listas, paneles).
- **Por qué TypeScript:** tipado estricto que evita errores en tiempo de ejecución, fundamentales en un sistema con roles y datos sensibles.
- **Por qué Vite:** arranque y recarga en caliente (HMR) muy rápidos, comparado con alternativas más pesadas.

Firebase (Auth + Firestore + Hosting)

- **Por qué una BBDD NoSQL en la nube:** la app necesita **tiempo real** (las asistencias/novedades de una escuela deben verse al instante) y sincronización multi-dispositivo, que Firestore resuelve con `onSnapshot`.
- **Por qué Firestore y no un backend propio:** evita mantener servidores, autenticación y despliegue se simplifican, y el modelo de reglas de seguridad encaja con los roles.

react-hook-form + Zod

- Por qué: validación declarativa y schemas centralizados, evitando formularios con errores inconsistentes.

jsPDF + Recharts

- Reportes PDF de asistencias/historial y gráficos de actividad.

2. Cómo se instaló (paso a paso real)

2.1. Creación del proyecto base

```
# Crear proyecto Vite con plantilla de React + TypeScript
npm create vite@latest proyecto-pasantia -- --template react-ts
cd proyecto-pasantia

# Instalar dependencias base
npm install
```

Esto generó la estructura raíz inicial: `index.html`, `src/`, `vite.config.ts`, `tsconfig*.json`, `eslint.config.js`.

2.2. Instalación de dependencias de la aplicación

```
npm install react-router-dom firebase
npm install react-hook-form zod
npm install recharts jspdf jspdf-autotable
npm install lucide-react motion @formkit/auto-animate
npm install -D prettier @testing-library/react @testing-library/jest-dom
```

2.3. Configuración de TypeScript y Vite

- Se definió el **alias** `@/` → `src/` en `vite.config.ts` y `tsconfig.app.json`, para importaciones limpias.
- `tsconfig.app.json` CON `strict: true`.
- Se revisó `vite.config.ts` con el plugin de React y el de PWA (instalado luego para soporte de instalación móvil).

2.4. Configuración de ESLint + Prettier

- Flat config en `eslint.config.js` con `typescript-eslint`, reglas de React Hooks, y `eslint-plugin-prettier`.
- `.prettierrc`: single quotes, trailing comma es5, printWidth 100.

2.5. Configuración de Firebase

Ver detalle en `09_setup_firebase.md`. Resumen:

1. Crear proyecto en **Firebase Console**.
2. Habilitar **Authentication** (email/password).
3. Crear **Firestore** (modo de prueba en desarrollo).
4. Configurar variables de entorno en `.env` (API keys, projectId, etc.).
5. Inicializar la app en `src/services/firebase.ts` (`initializeApp`, `getAuth`, `initializeFirestore`).
6. Definir `firestore.rules` para restringir el acceso por `escuelaId` según el rol (los no-supervisores solo leen su propia escuela).

2.6. Configuración del router y estructura de carpetas

- `src/routes/index.tsx` con las rutas protegidas por rol (`ROUTE_PERMISSIONS` en `AuthContext.tsx`).
- Estructura por carpetas: `components/`, `context/`, `hooks/`, `pages/`, `services/`, `types/`, `utils/`.

3. Documentación para entender el proyecto

Además de configurar el entorno, en esta primera semana se realizó la **documentación inicial** para entender de qué se trataba el proyecto y qué había que hacer. Esta documentación quedó en `documentation/`:

- **Contexto general** (`01_contexto_general.md`): en qué consiste el sistema, para quién es y el problema que resuelve.
- **Requerimientos** (`02_requerimientos.md`): qué funcionalidades debía tener la aplicación.
- **Arquitectura** (`03_arquitectura.md`): cómo se estructura el sistema y las decisiones técnicas.
- **Reglas de negocio** (`04_reglas_negocio.md`): reglas de asistencias, novedades, incidentes y roles.

- **Base de datos** (`05_base_datos.md`): modelo de datos en Firestore (colecciones y campos).
- **API** (`06_api.md`): servicios y consultas de datos.
- **Diseño UI** (`07_diseno_ui.md`): pantallas, flujos y estilo visual.
- **Configuración de Firebase** (`09_setup_firebase.md`): pasos para Firebase.
- **Manejo de errores, validaciones, estado, testing, despliegue y accesibilidad** (docs 11 a 17).

Esta documentación sirvió de **guía** para saber qué construir a lo largo del proyecto y para retomarlo en cualquier momento (se documenta en `CONTEXT.md`).

4. Verificación inicial

Cada paso se validó con:

```
npx tsc -b --noEmit # compilación
npm run lint        # linting
npm run dev          # arranque local en http://localhost:5173
```

5. Tiempo invertido en la Semana 1

La siguiente tabla muestra el tiempo que se tardó en cada tarea de esta primera semana (documentación + instalación y configuración del entorno). Los valores están estimados en base a la jornada de trabajo del 23 y 24 de julio de 2026.

Tarea	Tiempo estimado
Documentación para entender el proyecto (contexto, requerimientos, arquitectura, base de datos, API, diseño UI, etc.)	~2 días
Elección y revisión del stack tecnológico	~1 h
Creación del proyecto Vite + React + TS	~30 min
Instalación de todas las dependencias	~20 min
Configuración TypeScript + alias <code>@/</code>	~30 min
Configuración ESLint + Prettier	~30 min
Configuración Firebase (proyecto, Auth, Firestore, <code>.env</code> , inicialización)	~1 h
Reglas de seguridad Firestore + índices	~1 h
Router protegido por rol + estructura de carpetas	~1 h
Total Semana 1	~2 días de trabajo

A esta base de instalación se le sumó, en la misma semana, el armado del **MVP** (tipos, servicios, componentes y el panel del Supervisor), que se detalla en `01_semana_1.md` .

6. Pendientes y observaciones

- Ajustar `firestore.rules` para el escenario definitivo de producción.
- Configurar variables de entorno para el despliegue (Netlify y/o Firebase Hosting).
- Las semanas siguientes de esta bitácora documentan las funcionalidades construidas sobre esta base.

02 - Base de Datos

Este documento de la bitácora recopila todo lo relacionado con la **base de datos** del proyecto: qué tecnología se usó, por qué, qué estructura tiene, cómo se relacionan los datos, las reglas de seguridad y los índices. Se trabajó a lo largo de varias semanas del proyecto.

1. Tecnología elegida y por qué

Se utilizó **Firestore** (base de datos NoSQL en la nube de Firebase).

Por qué Firestore

- **Modelo documento/colección:** encaja de forma natural con la estructura del sistema (escuelas, usuarios, registros de cada día).
- **Tiempo real:** necesario para que las asistencias, novedades e incidentes se vean al instante en los paneles del Supervisor y de cada escuela. Firestore lo resuelve con suscripciones `onSnapshot`.
- **Sincronización multi-dispositivo:** los datos se comparten entre todos los perfiles sin mantener un servidor propio.
- **Soporte offline:** escritura local y sincronización automática cuando vuelve la conexión (clave porque las escuelas pueden quedarse sin internet).
- **Seguridad por reglas:** se puede restringir el acceso por rol y por escuela directamente en la base de datos.
- **Ya integrado con Firebase Auth:** el mismo ecosistema maneja login y datos.

Por qué no una base SQL tradicional

- En SQL (tablas + relaciones con joins) se habría necesitado un servidor backend propio y más configuración de autenticación/sincronización.
- El volumen y la forma de consulta del sistema (registros por día por escuela) se adaptan mejor a documentos.
- Se simplifica el despliegue: no hay servidor de base de datos que administrar.

2. Estructura de la base de datos

2.1 Colecciones

```
firestore/  
├─ escuelas/           # Un documento por escuela  
├─ usuarios/           # Un documento por usuario del sistema  
├─ docentes/           # Un documento por docente (cargados por  
el Supervisor)  
├─ asistencias/         # Un documento por carga de asistencia de  
gestión  
├─ asistencia_docentes/ # Un documento por carga de asistencia de  
docentes  
├─ novedades/           # Un documento por cada novedad registrada  
├─ incidentes/          # Un documento por cada incidente  
registrado  
└─ fotos/              # Un documento por cada foto de planilla  
subida
```

2.2 escuelas/{schoolId}

Campo	Tipo	Descripción	Obligatorio
nombre	string	Nombre de la escuela	Sí
turno	string	Turno (mañana, tarde, noche)	Sí
direccion	string	Dirección de la escuela	No
activa	boolean	Si la escuela está activa en el sistema	Sí

2.3 usuarios/{userId}

Campo	Tipo	Descripción	Obligatorio
nombre	string	Nombre completo del usuario	Sí
email	string	Correo (usado para login)	Sí
rol	string	Rol: director, vice, preceptor, secretario, conserje, supervisor	Sí
escuelaId	string	Referencia al documento de escuela	Sí
cargo	string	Cargo específico del usuario	Sí
activo	boolean	Si el usuario está activo	Sí
createdAt	timestamp	Fecha de creación	Sí

2.4 asistencias/{attendanceId}

Campo	Tipo	Descripción	Obligatorio
escuelaId	string	Referencia a la escuela	Sí
fecha	timestamp	Fecha del registro	Sí
cargadoPor	string	UID del usuario que cargó	Sí
cargadoPorNombre	string	Nombre del usuario que cargó	Sí
registros	array	Asistencia de cada integrante (nombre, cargo, presente, motivo)	Sí
createdAt	timestamp	Fecha de creación	Sí
verificada	boolean	Si el Supervisor la verificó	No
verificadoPor / verificadoPorNombre / verificadoEn	-	Datos de la verificación	No

2.5 asistencia_docentes/{attendanceId}

Igual que `asistencias` pero para docentes. Cada elemento de `registros[]` lleva `nombre`, `materia`, `presente`, `motivo`.

2.6 docentes/{docenteId}

Campo	Tipo	Descripción	Obligatorio
nombre	string	Nombre del docente	Sí
materia	string	Materia o área	No
escuelaId	string	Referencia a la escuela	Sí
activo	boolean	Solo los activos aparecen en el formulario	Sí
createdAt	timestamp	Fecha de creación	Sí

2.7 novedades/{newsId}

Campo	Tipo	Descripción	Obligatorio
escuelaId	string	Referencia a la escuela	Sí
fecha	timestamp	Fecha de la novedad	Sí
tipo	string	acto, actividad, suspension, evento, otro	Sí
hora	string	Hora (HH:MM)	No
descripcion	string	Descripción	Sí
cargadoPor / cargadoPorNombre	string	Quién registró	Sí
createdAt	timestamp	Fecha de creación	Sí

2.8 incidentes/{incidentId}

Campo	Tipo	Descripción	Obligatorio
escuelaId	string	Referencia a la escuela	Sí
fecha	timestamp	Fecha del incidente	Sí
categoria	string	rotura, filtracion, falla_servicio, urgencia, seguridad, otro	Sí
urgencia	string	baja, media, alta	Sí
ubicacion	string	Lugar dentro de la escuela	No
fotoDataUrl	string	Foto comprimida en base64	No
descripcion	string	Descripción	Sí
estado	string	pendiente, en_analisis, en_gestion, resuelto	Sí
cargadoPor / cargadoPorNombre	string	Quién registró	Sí
createdAt / updatedAt	timestamp	Creación y última actualización de estado	Sí

Historial de cambios de estado: los cambios de `estado` se registran como historial de trazabilidad (cada vez que el Supervisor cambia el estado, queda constancia de cuándo y quién lo hizo). Se agregó en la Semana 4.

2.9 fotos/{fotoId}

Campo	Tipo	Descripción	Obligatorio
escuelaId	string	Referencia a la escuela	Sí
fecha	string	Fecha YYYY-MM-DD	Sí
dataUrl	string	Imagen comprimida en base64 (JPEG ~1024px)	Sí
nombreArchivo	string	Nombre original del archivo	Sí
subidoPor / subidoPorNombre	string	Preceptor que subió	Sí
createdAt	timestamp	Fecha de creación	Sí

Nota sobre fotos: las imágenes se guardan **comprimidas en base64 dentro del documento** (sin Firebase Storage), redimensionadas a ~1024px con calidad ~0.6 para no superar el límite de 1 MiB por documento.

3. Por qué las decisiones de diseño

- **escuelaId como clave de relación:** cada registro (asistencia, novedad, incidente, foto, docente, usuario) referencia a su escuela mediante `escuelaId`. Esto permite que cada perfil solo acceda a los datos de su propia escuela y que el Supervisor vea todo.
- **Guardar `cargadoPor` + `cargadoPorNombre`:** mantener el UID y el nombre evita consultas extra y permite mostrar “quién cargó” de forma directa.
- **activo (boolean) en usuarios y docentes:** en lugar de borrar registros, se desactivan. Así se conserva el historial y solo aparecen activos en los formularios.
- **Fotos en base64:** evita depender de Firebase Storage y simplifica el alta/lectura; a cambio exige compresión para respetar límites.
- **Timestamp de auditoría en usuarios/docentes** (`createdAt`, y trail de auditoría para gestiones del Supervisor): trazabilidad de quién creó/quién editó.

4. Cómo se relaciona cada cosa

Firestore **no tiene joins nativos**: las relaciones se resuelven en el frontend mediante consultas separadas o al cargar los datos.

```

escuelas ← usuarios.escuelaId
escuelas ← docentes.escuelaId
escuelas ← asistencias.escuelaId
escuelas ← asistencia_docentes.escuelaId
escuelas ← novedades.escuelaId
escuelas ← incidentes.escuelaId
escuelas ← fotos.escuelaId

usuarios ← asistencias.cargadoPor
usuarios ← asistencia_docentes.cargadoPor
usuarios ← novedades.cargadoPor
usuarios ← incidentes.cargadoPor
usuarios ← fotos.subidoPor

```

Es decir: **escuelas es la “colección padre”** a la que apuntan casi todos los registros, y **usuarios identifica a la persona que realizó cada carga**.

5. Índices compuestos

Los índices permiten consultar registros combinando dos o más campos (por ejemplo, todas las asistencias de una escuela ordenadas por fecha). Los definidos en

firestore.indexes.json :

Colección	Campos indexados	Motivo
asistencias	escuelaId + fecha (desc)	Asistencias de una escuela ordenadas por fecha
asistencia_docentes	escuelaId + fecha (desc)	Asistencias docentes de una escuela
novedades	escuelaId + fecha (desc)	Novedades de una escuela por fecha
incidentes	escuelaId + fecha (desc)	Incidentes de una escuela por fecha
incidentes	estado + fecha (desc)	Filtrar incidentes por estado
fotos	escuelaId + fecha + createdAt (desc)	Fotos de una escuela por fecha/subida
fotos	escuelaId + createdAt (desc)	Listar fotos de una escuela

6. Reglas de seguridad (Firestore Rules)

Las reglas de `firestore.rules` controlan **quién puede leer y escribir cada colección** y qué campos puede modificar. Evolucionaron desde reglas básicas del MVP hasta reglas endurecidas por rol y por escuela.

Funciones de apoyo

- `isSignedIn()` : el usuario está autenticado.
- `hasProfile()` : el usuario tiene un perfil en `usuarios/{uid}` (existe el documento).
- `userProfile()` : obtiene el perfil del usuario.
- `isSupervisor()` : el perfil tiene rol `supervisor`.
- `userHasAnyRole(roles)` : el usuario tiene alguno de los roles indicados.
- `userBelongsToSchool(schoolId)` : el usuario pertenece a esa escuela.
- `canSeeSchool(schoolId)` / `canSeeSchoolData()` : el Supervisor ve todo; los demás solo su propia escuela.
- `onlyChangedFields(allowedFields)` : solo se permiten cambiar los campos indicados.

Permisos por colección

Colección	Lectura	Creación	Actualización	Eliminación
escuelas	Supervisor o miembro de la escuela	Solo Supervisor	Solo Supervisor	Solo Supervisor
usuarios	El propio usuario, Supervisor o mismo escuela	Solo Supervisor	El propio (solo nombre/email) o Supervisor	Solo Supervisor
asistencias	Datos de su escuela (o todo si Supervisor)	Roles director/vice/preceptor, de su escuela, con su UID como <code>cargadoPor</code>	Solo Supervisor (verificación)	No
asistencia_docentes	Ídem	Ídem	Solo Supervisor (verificación)	No
docentes	Datos de su escuela (o todo si Supervisor)	Solo Supervisor	Solo Supervisor	Solo Supervisor
novedades	Datos de su escuela	Roles director/vice, de su escuela	No	No
incidentes	Datos de su escuela	Roles director/vice, de su escuela	Solo Supervisor (solo <code>estado</code> y <code>updatedAt</code>)	No
fotos	Datos de su escuela	Solo preceptor, de su escuela, con su UID como <code>subidoPor</code>	No	Supervisor o el preceptor autor de escuela

Detalles clave de las reglas

- **Los no-supervisores solo leen los datos de su propia escuela**
(`canSeeSchoolData` compara `resource.data.escuelaId` con `profile.escuelaId`).
- **Al crear** un registro de asistencia/novedad/incidente/foto, las reglas exigen que el `cargadoPor` / `subidoPor` sea el UID del usuario autenticado **y** que la `escuelaId` coincida con la suya. Esto impide que un usuario cargue datos en nombre de otra escuela.
- **Solo el Supervisor puede cambiar el estado** de un incidente y **únicamente** los campos `estado` y `updatedAt` (protección contra cambios arbitrarios).
- **El Supervisor puede editar escuelas, docentes y crear usuarios.**
- **Eliminación** de datos sensibles (asistencias, novedades, incidentes) está **bloqueada** (`allow delete: if false`) para conservar el historial.

7. Tiempo estimado por tarea

La base de datos se construyó y fue evolucionando a lo largo del proyecto. Se lista el **tiempo estimado** de cada tarea relacionada (los valores son estimaciones del tiempo efectivo de trabajo).

Tarea	Cuándo	Tiempo estimado
Diseño inicial del esquema (colecciones, campos, relaciones)	Semana 1	~1 día
Documentación de la base de datos (05_base_datos.md)	Semana 1	~3-4 h
Definición de tipos TypeScript (Attendance, News, Incident, Foto, etc.)	Semana 1-2	~3 h
Consultas y servicios Firestore (queries básicas)	Semana 1	~3 h
Endurecimiento de reglas de seguridad (por rol y escuela)	Semana 2	~1 día
Verificación de asistencias por el Supervisor (campos verificada*)	Semana 2	~3 h
Campos ampliados en novedades/incidentes + fotos en base64	Semana 2	~4 h
Asistencia de docentes (asistencia_docentes)	Semana 2	~3 h
Índices compuestos y resolución de orderBy	Semanas 1-2	~2 h
Historial de cambios de estado de incidentes (trazabilidad)	Semana 4	~4 h
Queries jurisdiccionales (alcance Supervisor) y backup CSV	Semana 4	~4 h
Audit trail de usuarios/docentes y protección exists() en reglas	Semana 5	~4 h
Total aproximado	-	~6 días

8. Pendientes y observaciones

- Revisar reglas de seguridad en el escenario definitivo de producción.
- Evaluar si el volumen de fotos en base64 requiere migrar a Firebase Storage.
- Mantener los índices actualizados si se agregan nuevas consultas combinadas.

03 - Autenticación

Este documento de la bitácora recopila todo lo relacionado con la **autenticación del sistema**: cómo funciona, el porqué de las decisiones, el flujo de inicio de sesión, los roles y permisos, la creación de usuarios, el manejo de errores y la expiración de sesión.

1. Tecnología elegida y por qué

Se utiliza **Firebase Authentication** con el método **email + contraseña**.

Por qué Firebase Auth

- **Ya integrado con Firestore**: comparten el mismo ecosistema de Firebase, por lo que el usuario autenticado se asocia directamente con su perfil y sus datos en la base de datos.
- **Seguridad gestionada**: el hash de contraseñas, el control de intentos y la infraestructura de autenticación la maneja Firebase, sin implementar nada de eso desde cero.
- **Sesión persistente**: Firebase mantiene la sesión del usuario incluso al recargar la página, de forma transparente.
- **Reglas de Firestore**: las reglas de seguridad validan `request.auth` y el rol/perfil del usuario, así la autenticación y la autorización van de la mano.

Por qué email + contraseña

- Los usuarios del sistema son **cargados por el Supervisor** (no se registran por su cuenta), por lo que el alta con email y contraseña gestionada es el método más simple y controlado.

2. Cómo está organizada la autenticación

2.1. Inicialización de Firebase (`src/services/firebase.ts`)

- Se crea la app con `initializeApp(firebaseConfig)`.
- La configuración proviene de variables de entorno (`.env`):
`VITE_FIREBASE_API_KEY`, `VITE_FIREBASE_AUTH_DOMAIN`,
`VITE_FIREBASE_PROJECT_ID`, `VITE_FIREBASE_MESSAGING_SENDER_ID`,
`VITE_FIREBASE_APP_ID`.
- `auth = getAuth(app)` — instancia de autenticación.
- `db = initializeFirestore(...)` con caché local persistente y soporte multi-pestaña (permite offline y que varias pestañas compartan datos).

2.2. Estado global de autenticación

(`src/context/AuthContext.tsx`)

Un **Context de React** centraliza el estado de autenticación y lo exponen todos los componentes:

Valor	Descripción
user	Objeto <code>User</code> de Firebase (si hay sesión)
profile	Perfil del usuario desde Firestore (<code>usuarios/{uid}</code>)
isLoading	Si está cargando el estado inicial
isAuthenticated	Si hay sesión activa
login(email, password)	Inicia sesión
logout()	Cierra sesión
hasRole(...roles)	Comprueba si el usuario tiene alguno de esos roles
canAccess(route)	Valida si el usuario puede acceder a una ruta

2.3. Flujo de inicio de sesión (login)

1. El usuario envía email y contraseña desde `Login.tsx`.
2. `signInWithEmailAndPassword(auth, email, password)` valida las credenciales en Firebase.
3. Se **espera a cargar el perfil** desde Firestore (`usuarios/{uid}`) antes de resolver el login. Esto evita una condición de carrera en la que la app redirigía antes de tener el perfil (provocaba doble login / errores).
4. Se actualiza el estado: usuario, perfil, `isAuthenticated = true`.
5. La vista navega a la ruta principal.

2.4. Restauración de sesión (onAuthStateChanged)

- Al cargar la app, `onAuthStateChanged` detecta si hay una sesión guardada de Firebase y **restaura el estado automáticamente** (recarga de página no desconecta al usuario).
- Si hay usuario autenticado pero **no existe su perfil** en Firestore, se muestra una advertencia (un usuario puede tener cuenta en Auth pero faltar su documento en `usuarios`).

2.5. Cierre de sesión (logout)

- `signOut(auth)` termina la sesión en Firebase y el estado vuelve a `isAuthenticated = false`.
- El usuario es redirigido al login.

3. Roles y permisos

3.1. Roles del sistema

director | vice | preceptor | secretario | conserje | supervisor

3.2. Permisos por ruta (ROUTE_PERMISSIONS)

Ruta	Roles permitidos
/ (Home)	director, vice, preceptor, secretario, conserje, supervisor
/asistencia	director, vice, preceptor
/asistencia-docentes	director, vice, preceptor
/historial	director, vice, preceptor
/fotos	preceptor
/novedades	director, vice
/incidentes	director, vice
/supervisor	supervisor

`canAccess()` busca la coincidencia más larga de la ruta y verifica que el rol del usuario esté permitido. El **Supervisor** tiene acceso total y administra las escuelas, mientras que director/vice/preceptor acceden solo a los módulos de su escuela.

4. Creación de usuarios (Supervisor)

Los usuarios no se registran solos: **el Supervisor los crea** desde el panel de usuarios.

4.1. El problema con Firebase Auth

Existe el problema de que `createUserWithEmailAndPassword` **inicia sesión automáticamente** con el usuario recién creado. Si el Supervisor lo creara con la instancia de Auth principal, su sesión se vería reemplazada por la del usuario nuevo.

4.2. La solución: segunda app de Auth aislada

En `src/services/api/auth.ts` se crea una **segunda aplicación de Firebase** (`admin-user-creation`) con autenticación de **persistencia en memoria**:

```
const adminApp = initializeApp(firebaseConfig, 'admin-user-creation');
const adminAuth = initializeAuth(adminApp, { persistence: inMemoryPersist
```

- `createUserAccount(email, password)` crea el usuario en esa instancia aislada, captura su `uid` y **cierra la sesión temporal** al finalizar.
- Así la sesión del Supervisor en el Auth principal **queda intacta**.
- El `uid` obtenido se usa para crear el perfil del usuario en Firestore (`usuarios/{uid}`).

4.3. Restablecimiento de contraseña

- `sendPasswordReset(email)` envía un correo de restablecimiento de contraseña usando la instancia aislada (sin alterar la sesión del Supervisor).
- Está disponible desde el panel del Supervisor (para usuarios) y desde el perfil del usuario logueado.

5. Manejo de errores de autenticación

`src/utils/authErrors.ts` mapea los códigos de error de Firebase a **mensajes amigables en español**:

Código Firebase	Mensaje mostrado
auth/user-not-found	No existe una cuenta con ese email.
auth/wrong-password	La contraseña es incorrecta.
auth/invalid-email	El email no es válido.
auth/user-disabled	Esta cuenta fue desactivada.
auth/too-many-requests	Demasiados intentos. Esperá unos minutos.
auth/network-request-failed	Error de conexión. Revisá tu internet.
auth/invalid-credential	Email o contraseña incorrectos.
auth/email-already-in-use	Ya existe una cuenta con ese email.
auth/weak-password	La contraseña debe tener al menos 6 caracteres.
auth/operation-not-allowed	Este método de inicio de sesión no está habilitado.

Cualquier otro error se muestra de forma genérica, evitando exponer detalles técnicos al usuario final.

6. Expiración de sesión por inactividad

Se implementó un **cierre de sesión automático por inactividad** en

`AuthContext.tsx`:

- **Solo aplica al rol director**: tras **10 minutos sin actividad** (mouse, teclado, toque, scroll o clic) se cierra la sesión.
- **El Supervisor mantiene la sesión abierta**: pensado para que permanezca logueado en el panel de supervisión.
- El temporizador se **reinicia en cada actividad** del usuario y se limpia correctamente al desmontar el efecto.

Esto responde a un requerimiento de seguridad: la sesión del director no debe quedar abierta indefinidamente en una computadora compartida.

7. Verificación y validación

- Cada cambio en la autenticación se validó con: `npx tsc -b --noEmit`, `npm run lint` y `npm run test`.
- Existen **tests de Login** que cubren el flujo de inicio de sesión, el spinner de carga y los mensajes de error.

8. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Configuración inicial de Firebase Auth (email/password) y .env	Semana 1	~1 h
Estado global de autenticación (AuthContext con login/logout/perfil)	Semana 1	~4 h
Carga del perfil desde Firestore y asociación usuario-perfil	Semana 1	~2 h
Roles y permisos por ruta (ROUTE_PERMISSIONS , canAccess)	Semana 1	~2-3 h
Pantalla de Login (MVP)	Semana 1	~3 h
Mapeo de errores Firebase a mensajes amigables	Semana 3	~1-2 h
Corrección de condición de carrera en el login (doble login)	Semana 3	~2 h
Editar usuarios y restablecer contraseña desde el Supervisor	Semana 3	~3 h
Creación de usuarios con segunda app de Auth aislada	Semanas 3-5	~2 h
Carga del perfil vía onAuthStateChanged y advertencias de perfil faltante	Semana 5	~2 h
Cierre de sesión por inactividad (rol director)	Semana 5	~1-2 h
Login responsive (split-screen escritorio, full-screen móvil)	Semana 5	~1 día
Total aproximado	-	~4 días

9. Pendientes y observaciones

- Revisar la política de expiración de contraseñas si se requiere a futuro.
- Evaluar agregar **autenticación de dos factores** o validación por correo institucional si el ámbito lo requiere.
- Mantener sincronizados los permisos de rutas con las reglas de Firestore.

04 - Interfaz Visual, Formularios y Navegación

Este documento de la bitácora recopila todo lo relacionado con la **parte visual** del sistema: el sistema de temas, los componentes de interfaz, la navegación (barra superior, drawer, barra inferior), los formularios, los botones, los estados de carga/vacío y las **animaciones**.

1. Enfoque visual y por qué

La interfaz se construyó con **React + CSS propio** (sin frameworks de UI como Tailwind o Material UI), usando **variables CSS** como base del diseño.

Por qué CSS propio con variables

- **Ligereza y control total:** no se carga una librería de estilos ni clases predefinidas; cada componente tiene su propio CSS junto a su `.tsx`.
- **Tema unificado:** las **variables CSS** (`--primary-color`, `--surface-color`, `--text-color`, etc.) permiten cambiar el color de toda la app desde un solo lugar.
- **Consistencia:** el diseño usa una misma paleta, radios de borde, sombras y espaciados definidos como tokens reutilizables.

Por qué iconografía con lucide-react

- Biblioteca de iconos liviana y consistente con todo el sistema (cada botón, enlace y estado usa el mismo estilo de trazo).

Por qué `motion` y `auto-animate`

- `motion` para animaciones declarativas (entradas, transiciones de página, lightbox).
- `@formkit/auto-animate` para animar **listas y colecciones** de forma automática (inserción/eliminación de elementos).

2. Sistema de temas (claro / oscuro + color primario)

La app permite personalizar la apariencia en `/tema`.

Cómo funciona

- Las variables CSS base se definen en `global.css` en `:root`:
 - Colores: `--primary-color`, `--primary-light`, `--secondary-color`, `--background-color`, `--surface-color`, `--text-color`, `--text-secondary`, `--border-color`.
 - Sombras: `--shadow-xs` ... `--shadow-xl`.
 - Radios: `--radius-xs` ... `--radius-full`.
 - Espaciados: `--space-1` ... `--space-8`.
 - **Acentos por estado:** `--accent-green-*`, `--accent-blue-*`, `--accent-red-*`, `--accent-yellow-*` (éxito, info, error, advertencia).
 - **Tonos derivados:** `--primary-tint`, `--primary-tint-strong` y `gradient-accent` se calculan automáticamente a partir del color principal con `color-mix()`.
- `src/utils/theme.ts` manage el tema:
 - `ThemeState` guarda `primary`, `primaryLight` y `mode` (`light / dark`).

- `applyTheme()` fija las variables CSS en `document.documentElement`.
- Se **persiste en `localStorage`** (clave `sipnam-theme`) para que sobreviva a recargas.
- El modo oscuro aplica un mapa de variables oscuras (fondos, superficies, textos y acentos).
- `useTheme()` es el hook que exponen los componentes: `isDark`, `toggleMode()`, `setTheme()`.

El color primario se puede cambiar

- En `/tema` el usuario puede elegir otro color principal y la app recalcula **automáticamente** la paleta derivada (tints y gradientes). Esto permite dar identidad visual a cada escuela/organismo.

3. Navegación

3.1 Barra superior (Navbar)

- **Marca SIPNAM** a la izquierda.
- A la derecha: **campana de notificaciones**, botón de **menú (hamburguesa)**.
- **Barra de escritorio** (`navbar__desktop`): enlaces con iconos que se muestran **según el rol** (via `hasRole`):
 - Inicio, Asistencia, Docentes, Historial, Fotos, Novedades, Incidentes, Supervisión, Usuarios, Ayuda.
 - Búsqueda global (Ctrl+K), Mi Perfil y botón Salir.
- **Drawer (menú lateral)**: se abre con la hamburguesa, muestra los datos del usuario (avatar, nombre, rol), los enlaces según rol, el **toggle de tema oscuro** y el botón de cerrar sesión. Se cierra al hacer clic fuera o con Escape, y bloquea el scroll de fondo mientras está abierto.
- Los enlaces identifican el **estado activo** (`isActive`) y usan `viewTransition` para la transición de página.

3.2 Barra de navegación inferior móvil (BottomNav)

- Pensada para **teléfonos**: barra fija abajo con acceso rápido a las secciones principales.
- Incluye un **indicador deslizante** que marca la pestaña activa y **badges** (por ejemplo, incidentes abiertos o asistencia pendiente).
- Fue parte de la conversión de la app a PWA (instalable en móvil).

4. Formularios

Todos los formularios del sistema siguen un patrón consistente con **react-hook-form + Zod**:

Cómo se construyen

1. Se define un **schema Zod** (`src/utils/validation.ts`) que valida tipos, obligatoriedad, rangos y mensajes de error.
2. `useForm` con `zodResolver(schema)` conecta el formulario con la validación.
3. Los campos usan `register` (`inputs/selects`) y `Controller` (para componentes como `DatePicker`).
4. `formState.errors` muestra los mensajes de validación junto a cada campo.
5. En el envío se maneja el estado `isSubmitting`, se muestra **feedback de éxito o error** y (si corresponde) una **animación de éxito**.

Características comunes a todos los formularios

- **Validación en tiempo real** con mensajes de error en español.
- **Contador de caracteres** en campos de texto largo (ej. descripción 0/500).
- **Auto-guardado en borrador** (`useFormDraft`): si el usuario se va sin enviar, el contenido se conserva en `localStorage` y se recupera al volver.
- **Feedback háptico** (`useHaptic`) al enviar correctamente en dispositivos con vibración.
- **Feedback offline**: si no hay conexión, el registro se guarda localmente y se avisa que se sincronizará al volver.
- **Mensajes de error amigables** de Firestore (`friendlyFirestoreError`).

Ejemplos de formularios

- **Novedades** (`pages/Novedades`): fecha, tipo, hora y descripción, con hint contextual y animación de éxito (confeti).
- **Incidentes** (`pages/Incidentes`): categoría, urgencia, ubicación, foto y descripción.
- **Asistencia de gestión** (`AttendanceForm`): lista de integrantes con presente/ausente y motivo.
- **Asistencia de docentes**: foto diaria + lista de docentes.
- **Usuarios/Escuelas (Supervisor)**: alta y edición con formularios react-hook-form + Zod.

5. Componentes de interfaz reutilizables

En `src/components/common/` hay componentes que se usan en todas las pantallas:

Componente	Función
<code>Button</code>	Botón con variantes (primario, secundario, peligro, ghost), estados de carga (spinner) y éxito (check)
<code>Navbar</code> / <code>BottomNav</code>	Navegación (escritorio + móvil)
<code>GlobalSearch</code>	Búsqueda global con atajo Ctrl+K
<code>NotificationBell</code>	Campana de notificaciones con actualización en tiempo real
<code>DatePicker</code>	Selector de fecha
<code>StatusBadge</code>	Insignia de estado (pendiente, resuelto, etc.)
<code>EmptyState</code>	Estado vacío con ilustración, mensaje y botón de acción
<code>Skeleton</code>	Placeholders de carga animados
<code>LoadingScreen</code>	Pantalla de carga (splash)
<code>ConfirmDialog</code>	Diálogo de confirmación animado
<code>ErrorBoundary</code>	Captura errores de renderizado y los muestra de forma amigable
<code>Breadcrumb</code>	Migas de pan (navegación de contexto)
<code>Pagination</code>	Paginación de listas
<code>FilterBar</code>	Barra de filtros animada
<code>Timeline</code>	Línea de tiempo (ej. historial de incidentes)
<code>SwipeableRow</code>	Filas con gesto de deslizar
<code>IncidentHistory</code>	Historial de cambios de estado de incidentes
<code>WelcomeTour</code>	Recorrido guiado según rol al primer ingreso
<code>ChangelogModal</code>	Novedades de la versión al primer acceso tras actualizar
<code>SuccessAnimation</code>	Animación de éxito (confeti)

6. Estados de carga y vacío

- **Skeleton loadings:** al cargar datos, las pantallas muestran placeholders animados (Home, Historial, Fotos, Supervisor, etc.) para que la interfaz no “salte” cuando llegan los datos. Se hace un **crossfade** del skeleton al contenido real.
- **Empty states:** cuando no hay datos, se muestra una ilustración con mensaje y, en muchos casos, un **botón de acción** (ej. “Registrar primera novedad”) y una **animación de entrada**.

7. Animaciones

Librerías y técnicas usadas

- `motion` : animaciones declarativas de componentes (entrada, salida, zoom, lightbox).
- `@formkit/auto-animate` : anima listas y colecciones automáticamente (inserciones/eliminaciones en grids de escuelas, listas de actividad).
- **View Transitions API** (`viewTransition` en React Router): transición fluida al **navegar entre páginas** y **morph** de la tarjeta de escuela hacia su detalle.
- **CSS transitions/keyframes:** animaciones de entrada, cruce (crossfade), el indicador deslizable de la barra inferior y la **aurora** de fondo.
- `useCountUp` : contadores animados (estadísticas).
- `useAmbientMotion` : animaciones ambientales de fondo según el estado de conexión (Wi-Fi).

Dónde se aplican

Animación	Componente / pantalla
Splash / loading screen	Carga inicial
Toasts (entrada/salida)	Sistema de notificaciones
View transitions entre rutas	Toda la navegación
Morph tarjeta → detalle	Escuelas (Supervisor)
Grid de escuelas (insert/delete)	Supervisor
Acordeón (expand/collapse)	Detalle de escuela
Confeti de éxito	Formulario de Novedades
Mapa de calor	Home del Supervisor
Crossfade skeleton → contenido	Screens principales
Indicador deslizable + badges	Barra inferior móvil
Aurora animada	Fondo (ambient)

8. Accesibilidad y responsive

- **Focus-visible rings** y `backdrop-blur` en modales (accesibilidad de teclado).
- **ARIA labels** en botones de solo icono (changelog, close, etc.).
- **Responsive:** el layout se adapta a escritorio (login split-screen, barra superior con enlaces), tablet y móvil (login full-screen, drawer + barra inferior).
- **Contraste de tema oscuro** ajustado (fondos, superficies y textos del mapa oscuro).
- **CSS puro** para el WelcomeTour (en lugar de `motion`) para asegurar que se muestre bien.

9. Verificación

- Lint: `npm run lint`, compilación: `npx tsc -b --noEmit`, tests: `npm run test`.

- Existen tests de componentes de UI (StatusBadge, AttendanceRow, Button implícito en Login, etc.).

10. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Estructura de CSS con variables y design tokens	Semana 1	~1 día
Pantalla de Login (MVP)	Semana 1	~3 h
Navegación: Navbar + acceso según rol	Semana 1	~4 h
Componentes reutilizables base (Button, inputs, listas)	Semana 1	~4 h
Sistema de tema claro/oscuro + configuración en <code>/tema</code>	Semana 3	~1 día
Skeletons de carga	Semana 3-4	~3 h
Estilos responsive (escritorio/tablet/móvil)	Semanas 3-5	~1 día
Formularios con react-hook-form + Zod	Semanas 1-2	~1-2 días
Auto-guardado en borrador (<code>useFormDraft</code>)	Semana 5	~3 h
Barra inferior móvil (<code>BottomNav</code>) con badges	Semana 4	~3 h
Animaciones (motion, auto-animate, view transitions)	Semana 4	~1-2 días
Animaciones ambientales (aurora, contadores)	Semanas 4-5	~4 h
Login responsive (split-screen / full-screen)	Semana 5	~1 día
Accesibilidad (focus rings, ARIA)	Semanas 4-5	~4 h
Total aproximado	-	~10-12 días

11. Pendientes y observaciones

- Revisar contraste de color en todos los modos/temas personalizados.
- Ampliar cobertura de tests de componentes de UI.
- Evaluar agregar theme presets para cada escuela.

05 - Reglas de Negocio

Este documento de la bitácora recopila las **reglas de negocio** del sistema: qué se puede hacer, quién puede hacerlo y bajo qué condiciones. Fueron la guía para implementar los formularios, la navegación, las reglas de Firestore y los roles.

Las reglas se confirman y se hacen cumplir tanto en el **frontend** (validación de formularios, rutas por rol) como en la **base de datos** (reglas de seguridad Firestore).

1. Reglas de asistencia de gestión

ID	Regla
BR-AS-01	Solo director, vice-director o preceptor pueden completar el formulario de asistencia.
BR-AS-02	El director puede cargar un solo formulario por día por su escuela.
BR-AS-03	El vice-director puede cargar un solo formulario por día por su escuela.
BR-AS-04	Cada preceptor puede cargar un formulario por día (si hay 3 preceptores, hasta 3 formularios/día en total).
BR-AS-05	El formulario es de carga masiva : se registra la asistencia de todos los integrantes de la gestión (director, vice, preceptores, secretarios, conserjes) en un solo envío.
BR-AS-06	Para cada integrante, el estado es obligatorio (presente o ausente).
BR-AS-07	Si un integrante está ausente , el motivo es obligatorio . Si está presente , el campo de motivo se oculta.

Cómo se hace cumplir

- **Duplicado por día:** antes de guardar se verifica (con `checkDuplicate`) si ya existe un registro de esa escuela para esa fecha. Si existe, se bloquea con el mensaje "Ya cargaste la asistencia de esta escuela para esa fecha."
- **Motivo obligatorio si ausente:** validación en el formulario (`AttendanceForm`) y para los integrantes sin nombre en modo secciones también se exige el nombre.

2. Reglas de novedades

ID	Regla
BR-NO-01	Solo director o vice-director pueden registrar novedades.
BR-NO-02	Los campos escuela, fecha y descripción son obligatorios.
BR-NO-03	No hay límite de novedades por día.

Cómo se hace cumplir

- Rutas/roles (`ROUTE_PERMISSIONS`) y reglas de Firestore: `novedades` solo se crean por roles `director` / `vice` de la propia escuela.
- Schema Zod (`novedadSchema`) exige tipo, fecha y descripción; la descripción tiene máximo 500 caracteres.

3. Reglas de incidentes

ID	Regla
BR-IN-01	Solo director o vice-director pueden registrar incidentes.
BR-IN-02	Los campos escuela, fecha y descripción son obligatorios.
BR-IN-03	Todo incidente nuevo se crea con estado "pendiente" por defecto.
BR-IN-04	Solo el Supervisor puede cambiar el estado de un incidente.
BR-IN-05	Transiciones de estado permitidas:
BR-IN-06	No hay límite de incidentes por día.

```
pendiente → en_analisis → en_gestion → resuelto
           ↓
           pendiente (se vuelve a pendiente si se requiere más
información)
```

Cómo se hace cumplir

- En Firestore (`firestore.rules`), incidentes solo se **crean** por roles `director / vice` de la propia escuela, y el `update` del estado (campos `estado` y `updatedAt`) **solo lo puede hacer el Supervisor**.
- Cada cambio de estado queda registrado en el **historial de trazabilidad** del incidente (quién y cuándo).

4. Reglas de autenticación y acceso

ID	Regla
BR-AU-01	Todo usuario debe estar autenticado para acceder a cualquier vista del sistema.
BR-AU-02	El rol del usuario se obtiene de Firestore al login y se guarda en el <code>AuthContext</code> .
BR-AU-03	Cada ruta protegida verifica el rol antes de renderizar; si no tiene permisos, se redirige a <code>/</code> .
BR-AU-04	Un usuario no puede modificar su propio rol ni el de otro usuario.

Cómo se hace cumplir

- `AuthContext.canAccess(route) + ROUTE_PERMISSIONS` .
- En Firestore, el `update` de `usuarios` solo permite cambiar `nombre / email` (no el rol) salvo que sea el Supervisor (`onlyChangedFields(['nombre', 'email'])`).

5. Reglas del Supervisor

ID	Regla
BR-SU-01	El Supervisor solo visualiza información salvo lo indicado; no registra asistencias, novedades ni incidentes.
BR-SU-02	El Supervisor puede ver la información de todas las escuelas de la jurisdicción.
BR-SU-03	El Supervisor puede cambiar el estado de los incidentes (gestión de casos) y marcar/desmarcar la verificación de asistencias.
BR-SU-04	El Supervisor no puede editar ni eliminar registros de asistencia, novedades o incidentes.

Cómo se hace cumplir

- En Firestore, el Supervisor tiene acceso de lectura a todas las escuelas (`canSeeSchool`).
- El `update` de `asistencias / asistencia_docentes` solo lo hace el Supervisor y únicamente en los campos de **verificación** (`verificada` , `verificadoPor` , `verificadoPorNombre` , `verificadoEn`).
- El `update` de `incidentes` solo lo hace el Supervisor y solo en `estado / updatedAt` .
- La **eliminación** está bloqueada (`allow delete: if false`) para conservar el historial.

6. Verificación de asistencias por el Supervisor

- El Supervisor puede **verificar** (validar) una asistencia de gestión o de docentes: se marca con `verificada: true` y se guardan los datos de quién y cuándo la verificó.
- Esto agrega una capa de **control de calidad** sobre las cargas realizadas por directores/vices/preceptores.

7. Roles del sistema (resumen)

director | vice | preceptor | secretario | conserje | supervisor

Rol	Principal responsabilidad
Director	Carga asistencia, novedades e incidentes
Vice	Carga asistencia, novedades e incidentes
Preceptor	Carga asistencia de gestión, docentes y foto diaria
Secretario / Conserje	Son registrados en la asistencia (no cargan formularios)
Supervisor	Supervisa todas las escuelas, gestiona estados de incidentes y verifica asistencias

8. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Definición de reglas de asistencia y formulario de carga masiva	Semana 1-2	~1 día
Reglas de novedades e incidentes	Semana 1-2	~4 h
Verificación de asistencias por el Supervisor	Semana 2	~3 h
Trazabilidad de cambios de estado de incidentes	Semana 4	~4 h
Refuerzo de reglas en Firestore según rol/escuela	Semana 2 y 5	~1 día
Total aproximado	-	~3-4 días

9. Pendientes y observaciones

- Revisar si el Supervisor debe poder editar registros en algún caso especial.
- Definir política de auditoría completa (quién modificó qué y cuándo) a futuro.

06 - Servicios y Consultas de Datos (Firestore)

Este documento de la bitácora recopila la **capa de servicios de datos** del sistema: todas las funciones que leen y escriben en Firestore y las suscripciones en **tiempo real**. Todo vive en `src/services/api/firestore.ts`.

1. Por qué una capa de servicios centralizada

- **Un solo punto de acceso a los datos:** todos los componentes llaman a funciones de `firestore.ts` en lugar de escribir consultas Firestore dispersas.
- **Tipado:** cada función devuelve el tipo correspondiente (`School` , `Attendance` , `Incident` , etc.), lo que evita errores.
- **Consistencia:** query constraints (fechas de "hoy", límites, orden) se definen una sola vez y se reutilizan.
- **Mantener índices en orden:** los índices compuestos requeridos por las combinaciones `where` + `orderBy` están documentados en el encabezado del archivo.

2. Estructura del servicio

Colecciones mapeadas

Constante	Colección Firestore
<code>schools</code>	<code>escuelas</code>
<code>users</code>	<code>usuarios</code>
<code>attendances</code>	<code>asistencias</code>
<code>news</code>	<code>novedades</code>
<code>incidents</code>	<code>incidentes</code>
<code>docentes</code>	<code>docentes</code>
<code>docenteAttendances</code>	<code>asistencia_docentes</code>
<code>fotos</code>	<code>fotos</code>

Dos modos de acceso

1. **Consultas de una sola lectura / escritura** (funciones `async` con `getDocs` / `getDoc` / `addDoc` / `updateDoc` / `deleteDoc`).
2. **Suscripciones en tiempo real** (funciones `subscribe*` con `onSnapshot`): cada vez que cambia un documento de la consulta, se notifica al callback y la interfaz se actualiza al instante.

3. Servicios por dominio

3.1 Escuelas

Función	Descripción
<code>getSchools()</code>	Lista todas las escuelas
<code>getSchoolById(id)</code>	Obtiene una escuela por id
<code>addSchool(data)</code>	Crea una escuela (Supervisor)
<code>updateSchool(id, data)</code>	Edita una escuela (Supervisor)
<code>deleteSchool(id)</code>	Elimina una escuela (Supervisor)

3.2 Usuarios

Función	Descripción
<code>getUsersBySchool(schoolId)</code>	Usuarios de una escuela
<code>getAllUsers()</code>	Todos los usuarios (Supervisor)
<code>addUserProfile(...)</code>	Crea perfil de usuario (Supervisor)
<code>setUserActive(id, active)</code>	Activa/desactiva un usuario
<code>updateUserProfile(id, data)</code>	Edita un usuario

3.3 Asistencia de gestión

Función	Descripción
<code>addAttendance(dto)</code>	Guarda una asistencia
<code>setAttendanceVerified(id, verified, ...)</code>	Verificación por el Supervisor
<code>getAttendanceByUserAndDate(...)</code>	Asistencia de un usuario en una fecha (duplicados)
<code>getAttendancesBySchool(schoolId, start, end)</code>	Asistencias de una escuela en un rango
<code>getTodayAttendances()</code>	Asistencias de hoy (todas)
<code>getTodayAttendancesBySchool(id)</code>	Asistencias de hoy de una escuela
<code>getAllAttendancesBySchool(id)</code>	Todas las asistencias de una escuela
<code>getAllAttendances(start?, end?)</code>	Asistencias en rango (jurisdicción, Supervisor)

3.4 Asistencia de docentes

Función	Descripción
<code>addDocenteAttendance(dto)</code>	Guarda asistencia de docentes
<code>setDocenteAttendanceVerified(...)</code>	Verificación por el Supervisor
<code>getDocenteAttendanceByUserAndDate(...)</code>	Duplicados
<code>getDocenteAttendancesBySchool(...)</code>	Asistencias docentes de una escuela
<code>getAllDocenteAttendances(start?, end?)</code>	Asistencias docentes en rango (Supervisor)

3.5 Docentes

Función	Descripción
<code>getDocentesBySchool(id)</code>	Docentes de una escuela
<code>getAllDocentes()</code>	Todos los docentes (Supervisor)
<code>addDocente(dto)</code>	Crea docente (Supervisor)
<code>setDocenteActive(id, active)</code>	Activa/desactiva docente
<code>updateDocente(id, data)</code>	Edita docente (Supervisor)

3.6 Novedades

Función	Descripción
<code>addNews(dto)</code>	Registra una novedad
<code>getNewsBySchool(id, start, end)</code>	Novedades de una escuela en rango
<code>getTodayNews()</code>	Novedades de hoy
<code>getTodayNewsBySchool(id)</code>	Novedades de hoy de una escuela
<code>getAllNewsBySchool(id)</code>	Todas las novedades de una escuela
<code>getAllNews(start?, end?)</code>	Novedades en rango (Supervisor)

3.7 Incidentes

Función	Descripción
<code>addIncident(dto)</code>	Registra un incidente
<code>getIncidentsBySchool(id, start, end)</code>	Incidentes de una escuela en rango
<code>getTodayIncidents()</code>	Incidentes de hoy
<code>getTodayIncidentsBySchool(id)</code>	Incidentes de hoy de una escuela
<code>getRecentIncidents(max)</code>	Incidentes recientes
<code>getAllIncidents(start?, end?)</code>	Incidentes en rango (Supervisor)
<code>updateIncidentStatus(id, estado, ...)</code>	Cambia el estado (Solo Supervisor) + traza el historial

3.8 Fotos

Función	Descripción
<code>addFoto(dto)</code>	Sube una foto de planilla
<code>deleteFoto(id)</code>	Elimina una foto
<code>getFotosBySchoolAndDate(id, fecha)</code>	Fotos de una escuela en una fecha
<code>getFotosBySchool(id)</code>	Fotos de una escuela

4. Suscripciones en tiempo real (`subscribe*`)

Cada una recibe un callback y devuelve una función `unsubscribe` para cancelar la suscripción al desmontar el componente.

Función	Descripción
<code>subscribeTodayAttendances(cb)</code>	Asistencias de hoy en vivo
<code>subscribeTodayAttendancesBySchool(id, cb)</code>	Asistencias de hoy de una escuela en vivo
<code>subscribeTodayNews(cb)</code>	Novedades de hoy en vivo
<code>subscribeTodayNewsBySchool(id, cb)</code>	Novedades de hoy de una escuela en vivo
<code>subscribeTodayIncidents(cb)</code>	Incidentes de hoy en vivo
<code>subscribeTodayIncidentsBySchool(id, cb)</code>	Incidentes de hoy de una escuela en vivo
<code>subscribeRecentIncidents(max, cb)</code>	Incidentes recientes en vivo
<code>subscribeAttendancesBySchool(id, cb)</code>	Asistencias de una escuela en vivo
<code>subscribeNewsBySchool(id, cb)</code>	Novedades de una escuela en vivo
<code>subscribeIncidentsBySchool(id, cb)</code>	Incidentes de una escuela en vivo
<code>subscribeDocenteAttendancesBySchool(id, cb)</code>	Asistencias docentes de una escuela en vivo
<code>subscribeFotosBySchool(id, cb)</code>	Fotos de una escuela en vivo
<code>subscribeLast7DaysCounts(cb)</code>	Conteos diarios de los últimos 7 días (gráficos)
<code>subscribeLast30DaysAttendance(cb)</code>	Densidad de asistencia de los últimos 30 días (mapa de calor)

Patrón de una suscripción

```
export function subscribeTodayAttendances(callback) {
  const q = query(
    collection(db, COLLECTIONS.attendances),
    where('fecha', '>=', Timestamp.fromDate(startOfToday())),
    orderBy('fecha', 'desc'),
    limit(100)
  );
  return onSnapshot(q, (snap) => {
    callback(snap.docs.map((d) => ({ id: d.id, ...d.data() }) as Attendan
  }));
}
```

- Se filtra por `fecha >= hoy`, se ordena descendente y se limita a 100 registros.
- `onSnapshot` notifica cada vez que hay cambios (insert, update, delete).
- El id de cada documento se agrega al objeto devuelto (`{ id, ...data }`).

5. Consultas que requieren índices compuestos

Combinar `where` + `orderBy` en campos distintos exige **índices compuestos** creados en Firebase Console (documentados en `documentation/05_base_datos.md`):

Consulta	Índice
<code>getSchools</code>	<code>activa + nombre</code>
<code>getAttendancesBySchool</code>	<code>escuelaId + fecha</code> (range + orderBy)
<code>getNewsBySchool</code>	<code>escuelaId + fecha</code>
<code>getIncidentsBySchool</code>	<code>escuelaId + fecha</code>
<code>getDocenteAttendancesBySchool</code>	<code>escuelaId + fecha</code>
<code>getFotosBySchool</code>	<code>escuelaId + createdAt</code>

Si una consulta falla, la consola de Firestore sugiere el índice a crear.

6. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Definir colecciones y constante de mapeo	Semana 1	~ 1 h
Servicios de escuelas y usuarios (CRUD)	Semana 1	~4 h
Servicios de asistencias y asistencia docentes	Semana 1-2	~ 1 día
Servicios de novedades, incidentes y fotos	Semana 1-2	~ 1 día
Función <code>updateIncidentStatus</code> con historial/trazabilidad	Semana 4	~3 h
Queries jurisdiccionales (Supervisor, rango de fechas)	Semana 4	~4 h
Suscripciones en tiempo real (<code>onSnapshot</code>)	Semana 3	~ 1-2 días
Suscripciones de conteos (gráficos y mapa de calor)	Semana 5	~4 h
Índices compuestos y resolución de fallas	Semanas 1-3	~3 h
Total aproximado	-	~6-7 días

7. Pendientes y observaciones

- Revisar el límite de 100 registros por consulta y agregar paginación para vistas muy cargadas.
- Centralizar aún más la construcción de queries para evitar duplicación entre funciones `get*` y `subscribe*`.

07 - Tiempo Real y Modo Offline

Este documento de la bitácora recopila dos capacidades clave del sistema: los **datos en tiempo real** (las pantallas se actualizan al instante cuando cambia la base de datos) y el **modo offline** (la app funciona sin conexión y sincroniza al volver el internet).

1. Tiempo real

1.1 Por qué

- El Supervisor debe ver las asistencias, novedades e incidentes de todas las escuelas **en el momento** en que se cargan, sin recargar la página.
- Los directores/preceptores deben ver reflejados al instante los registros de su escuela.

1.2 Cómo funciona

Se usan **suscripciones en tiempo real** de Firestore mediante `onSnapshot`. En lugar de consultar una vez, la app se “suscribe” a una consulta y Firebase notifica cada vez que cambia un documento que cumple los criterios.

```
export function subscribeTodayAttendances(callback) {
  const q = query(
    collection(db, COLLECTIONS.attendances),
    where('fecha', '>=', Timestamp.fromDate(startOfToday())),
    orderBy('fecha', 'desc'),
    limit(100)
  );
  return onSnapshot(q, (snap) => {
    callback(snap.docs.map((d) => ({ id: d.id, ...d.data() })) as Attendances);
  });
}
```

- Cada función `subscribe*` devuelve una `unsubscribe`, que se llama al desmontar el componente para no dejar suscripciones abiertas (evita fugas de memoria y renders innecesarios).

1.3 Dónde se aplica

- **Home del Supervisor:** asistencias, novedades e incidentes del día en vivo.
- **Detalle de escuela (Supervisor):** datos de la escuela y sus registros.
- **Historial / listados:** asistencias, novedades, incidentes de una escuela en vivo.
- **Gráficos:** conteos de los últimos 7 días y densidad de asistencia de los últimos 30 días (mapa de calor).

1.4 Consideración de rendimiento

- En el **Home** se reemplazaron algunas suscripciones en tiempo real por **fetch periódico** para evitar renders excesivos y consumo de recursos al navegar entre pantallas. Es un equilibrio entre actualidad de los datos y rendimiento.

2. Modo offline

2.1 Por qué

- Las escuelas pueden quedarse **sin internet**. El sistema debe seguir permitiendo cargar asistencias, novedades e incidentes y sincronizarlos después.
- Es un requerimiento funcional y no funcional clave (RF-IN-03 y RNF-03 del módulo de incidentes).

2.2 Estrategia de Firestore offline

Firestore tiene **soporte offline nativo** en el SDK web:

1. Los datos leídos se **cachean** automáticamente.
2. Las **escrituras se hacen localmente** y se sincronizan cuando vuelve la conexión.
3. No hace falta implementar la sincronización manual.

2.3 Configuración actual (`src/services/firebase.ts`)

Se usa **caché local persistente** con soporte multi-pestaña:

```
export const db = initializeFirestore(app, {
  localCache: persistentLocalCache({
    tabManager: persistentMultipleTabManager(),
  }),
});
```

Nota: inicialmente se usaba `enableIndexedDbPersistence`, pero esta API está **deprecada**; se migró a `initializeFirestore` con `persistentLocalCache`.

2.4 Detección de conexión (`useOnlineStatus`)

Hook que expone si hay conexión escuchando los eventos `online / offline` del navegador a partir de `navigator.onLine`.

2.5 Marcador de escrituras offline (`src/utils/offlineQueue.ts`)

- `markOfflineWrite()` : se llama antes de guardar un registro sin conexión; guarda una marca en `localStorage`.
- `hasOfflineWrites()` : indica si hay escrituras guardadas localmente pendientes.
- `clearOfflineWrites()` : limpia la marca tras la sincronización.

2.6 Banner de conexión (`ConnectionBanner`)

- Si no hay conexión: muestra *"Sin conexión — Los cambios se guardarán y sincronizarán cuando haya internet."* con conteo de pendientes.
- Al volver la conexión: usa `waitForPendingWrites(db)` para esperar la sincronización y luego muestra *"Sincronizado correctamente"* durante unos segundos y limpia el marcador.

2.7 Feedback en los formularios

- **Asistencia, novedades e incidentes:** si el envío se hace sin conexión, muestran *"Sin conexión: guardado en el dispositivo. Se sincronizará automáticamente al volver internet."* y llaman a `markOfflineWrite()`.

3. Comportamiento esperado

Con conexión

```
graph TD; A[Usuario carga un registro] --> B[Firestore guarda localmente + envía al servidor]; B --> C[Datos disponibles en tiempo real (onSnapshot)]
```

Sin conexión

```
graph TD; A[Usuario carga un registro] --> B[Firestore guarda localmente (IndexedDB)]; B --> C[App muestra "Guardado localmente" + marca offline]; C -- "(al restablecerse la conexión)" --> D[Firestore sincroniza automáticamente (waitForPendingWrites)]; D --> E[App muestra "Sincronizado correctamente"]
```

4. Limitaciones del modo offline

Funcionalidad	Offline	Online
Crear asistencia	Sí	Sí
Crear novedad	Sí	Sí
Crear incidente	Sí	Sí
Ver registros propios	Sí (cacheados)	Sí
Cambiar estado de incidente	No	Sí (Solo Supervisor)
Ver todos los registros (Supervisor)	Cacheado	Sí en tiempo real

5. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Configuración de persistencia offline (localCache)	Semana 1	~1 h
Migración de <code>enableIndexedDbPersistence</code> deprecado	Semana 3	~1-2 h
Hook <code>useOnlineStatus</code> (detección de conexión)	Semana 3-4	~1 h
Suscripciones <code>onSnapshot</code> en paneles del Supervisor	Semana 3	~1-2 días
Suscripciones por escuela (Home de directores/preceptores)	Semana 3	~1 día
Banner de conexión + <code>waitForPendingWrites</code>	Semana 4-5	~3 h
Marcador de escrituras offline y feedback en formularios	Semana 4-5	~3 h
Indicador de cola offline con contador	Semana 5	~2 h
Optimización: fetch periódico en Home	Semana 3	~2 h
Total aproximado	-	~5-6 días

6. Pendientes y observaciones

- Revisar el comportamiento de sincronización en escenarios de conflicto (dos dispositivos editando lo mismo).
- Considerar un límite mayor que 100 registros o paginación para vistas muy cargadas en tiempo real.

08 - Notificaciones

Este documento de la bitácora recopila el **sistema de notificaciones** del proyecto: la campana de notificaciones, los toasts en tiempo real, las alertas en vivo del Supervisor y las **notificaciones nativas** del navegador/dispositivo.

1. Por qué notificaciones

- El **Supervisor** debe enterarse al instante de nuevos incidentes, asistencias y novedades de todas las escuelas, sin tener que revisar manualmente.
- Los **directores/preceptores** deben ver la actividad reciente de su propia escuela.

2. Componentes del sistema de notificaciones

Componente	Función
<code>NotificationBell</code>	Campana en la barra superior con conteo de no leídos y panel desplegable
<code>RealtimeNotifications</code>	Toasts en tiempo real para el Supervisor (nuevos registros)
<code>SupervisorLiveAlerts</code>	Alertas en vivo del Supervisor + notificaciones nativas opt-in
<code>ToastContext</code>	Sistema de toasts reutilizable

Todos se montan en `MainLayout`, de modo que están disponibles en toda la app logueada.

3. Campana de notificaciones (`NotificationBell`)

- Muestra un **badge con el número de notificaciones no leídas**.
- Al abrir, se marcan como leídas (el contador se pone en 0).
- **Según el rol:**
 - **Supervisor:** se suscribe a `subscribeRecentIncidents` y muestra los **incidentes pendientes** (no resueltos) de todas las escuelas. Al hacer clic navega al panel de supervisión.
 - **Escuelas (director/vice/preceptor):** se suscribe a `subscribeTodayAttendancesBySchool` y `subscribeTodayNewsBySchool` y muestra las asistencias y novedades cargadas hoy en su escuela.
- Las suscripciones se cancelan al desmontar el componente (`unsubscribe`).

4. Toasts en tiempo real (`RealtimeNotifications`)

- Componente **solo para el Supervisor**.
- Se suscribe a `subscribeTodayIncidents`, `subscribeTodayAttendances` y `subscribeTodayNews`.
- Al recibir datos, muestra un **toast** (aviso temporal) con el detalle del nuevo registro usando `ToastContext`.
- **Evita el "spam" inicial:** con un ref de inicialización, ignora el primer snapshot (que es el estado ya existente) y solo notifica **cambios posteriores**.
- Tipos de toast: `warning` (incidente) e `info` (asistencia/novedad).

5. Alertas en vivo del Supervisor (SupervisorLiveAlerts)

- Componente dedicado al Supervisor con dos funciones:

5.1 Alertas en pantalla

- Mantiene una lista de alertas visuales (asistencia, novedad, incidente) con iconos por tipo.
- Detecta **solo registros nuevos** comparando contra un set de ids ya vistos (seenRef).
- Si llegan varios a la vez, agrupa en "N registros nuevos de X".
- Las alertas se auto-eliminan después de un tiempo.

5.2 Notificaciones nativas (opt-in)

- Pide **permiso de notificaciones** del navegador (banner "opt-in" que desaparece si se descarta o acepta).
- Si el permiso está **otorgado** y la pestaña está **en segundo plano** (document.visibilityState !== 'visible'), muestra una **notificación nativa** del dispositivo:
 - A través del **service worker** (reg.showNotification), con fallback a new Notification .
 - Incluye icono y texto del nuevo registro.
- notificationsSupported() valida que el navegador soporte Notification .

6. Cómo se detectan solo los registros nuevos

Tanto RealtimeNotifications como SupervisorLiveAlerts usan el mismo patrón:

1. En el **primer snapshot** cargan los ids existentes en un set (sin notificar).
2. En los **snapshots siguientes** comparan los ids contra el set: los que no estaban son "nuevos".
3. Agregan los nuevos al set y notifican.

Esto evita notificar datos que ya estaban al iniciar la vista.

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Sistema de toasts (ToastContext)	Semana 4	~3 h
Campana de notificaciones (NotificationBell) con conteo	Semana 4	~4 h
Toasts en tiempo real para Supervisor (RealtimeNotifications)	Semana 5	~3 h
Alertas en vivo del Supervisor (SupervisorLiveAlerts)	Semana 5	~4 h
Notificaciones nativas opt-in (service worker)	Semana 5	~3 h
Detección de registros nuevos (sets de ids vistos)	Semana 5	~2 h
Total aproximado	-	~3 días

8. Pendientes y observaciones

- Considerar persistir el "leído/no leído" por usuario en Firestore (hoy el conteo es por sesión).
- Evaluar el envío de notificaciones **push** de Firebase Cloud Messaging para cuando la app esté cerrada.
- Revisar el manejo de permisos en iOS (PWA).

09 - Reportes y Exportación

Este documento de la bitácora recopila el **sistema de reportes y exportación** de datos del proyecto: exportaciones a **CSV**, a **PDF** y el **respaldo jurisdiccional** del Supervisor.

1. Por qué

- El Supervisor y las escuelas necesitan **reportes** de las cargas (asistencias, novedades, incidentes) para control, seguimiento y presentaciones.
- Se exportan datos de forma **portable** (CSV abre en Excel; PDF es imprimible/compartible).

2. Exportación a CSV (`src/utils/exportCsv.ts`)

Función genérica para descargar cualquier dataset como CSV.

`downloadCsv(filename, headers, rows)`

- Recibe un nombre de archivo, los encabezados y las filas.
- **Escapa correctamente** valores con comas, comillas, saltos de línea o punto y coma.
- Agrega **BOM UTF-8** (`\uFEFF`) para que Excel interprete bien los acentos.
- Genera un `Blob` y lo descarga con un enlace temporal (`URL.createObjectURL + link.click()`), limpiando la URL después.

Uso: exportar asistencias, novedades, incidentes e historial a Excel-compatible.

3. Exportación a PDF (`src/utils/pdfExport.ts`)

Usa **jsPDF** + **jspdf-autotable** para generar reportes profesionales.

`exportHistorialPDF(data)`

- Recibe asistencias, asistencias de docentes, novedades e incidentes (y rango de fechas opcional).
- **Encabezado:** logo/título "SIPNAM", subtítulo del sistema, título del reporte y filtros aplicados (desde/hasta).
- **Tablas:** usa `autoTable` para renderizar cada sección (asistencias, docentes, novedades, incidentes) con sus columnas.
- **Pie de página:** "SIPNAM - Exportado {fecha} - Página X/Y" en cada página.
- Convierte los valores enum a etiquetas legibles (tipo de novedad, categoría/urgencia de incidente) con los helpers de `constants.ts`.

4. Respaldo jurisdiccional del Supervisor (`src/utils/exportAll.ts`)

Permite al Supervisor **descargar todos los datos de la jurisdicción** (todas las escuelas) en CSV, en las colecciones correspondientes.

- `exportAll` (con `ExportAllOptions`):
 - Opcionalmente filtra por rango de fechas (`dateFrom / dateTo`).

- **Reporta progreso** vía `onProgress` (current/total/label) para mostrar una barra de avance en la UI.
- Genera un CSV por cada tipo de dato: asistencias, asistencias de docentes, novedades e incidentes, con detalle (ej. cada asistencia expande sus registros como "Nombre (P)" o "Nombre (A: motivo)").

Tarjeta de respaldo de datos

- En el panel del Supervisor hay una **tarjeta de respaldo de datos** que usa `exportAll` para exportar a nivel jurisdicción.
- Complementa el **banner de advertencia de purga de datos a fin de año**, recordando al Supervisor respaldar antes de esa fecha.

5. Impresión amigable

- Se agregaron **estilos específicos de impresión** (`@media print`) para que al imprimir/guardar como PDF desde el navegador, la salida sea limpia (sin ocultar elementos de UI como barra de navegación, botones, etc.).

6. Dónde se usan

Función	Ubicación
<code>downloadCsv</code>	Historial y panel del Supervisor (CSV)
<code>exportHistorialPDF</code>	Historial (exportar a PDF)
<code>exportAll</code>	Panel del Supervisor (respaldo jurisdiccional)

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Exportación CSV (<code>downloadCsv</code>) con escape y BOM	Semana 2	~2-3 h
Exportación de historial a PDF (<code>jsPDF</code> + <code>autoTable</code>)	Semana 4	~4-5 h
Respaldo jurisdiccional (<code>exportAll</code>) con progreso	Semana 4	~4 h
Tarjeta de respaldo de datos y banner de purga	Semana 4	~2-3 h
Estilos de impresión (<code>@media print</code>)	Semana 5	~2 h
Total aproximado	-	~2 días

8. Pendientes y observaciones

- Verificar el tamaño de PDFs muy grandes (muchas filas).
- Considerar exportar también **fotos** o generar un reporte consolidado por escuela.
- Evaluar exportación a Excel nativo (`.xlsx`) si se requiere más formato.

10 - Manejo de Errores

Este documento de la bitácora recopila cómo el sistema **captura y muestra los errores** de forma amigable para el usuario, tanto en la interfaz como en la capa de datos.

1. Estrategia general

- **Nunca mostrar errores técnicos crudos** al usuario final.
- Mapear los errores de Firebase/Firestore a **mensajes claros y accionables** en español.
- **Capturar errores de renderizado** para que un fallo en un componente no deje la app en blanco.
- Mostrar **feedback visual** (éxito/error) en los formularios con posibilidad de descartarlo.

2. Componentes del sistema de errores

Componente	Función
ErrorBoundary	Captura errores de renderizado de la app
friendlyFirestoreError()	Convierte errores de Firestore en mensajes amigables
getAuthErrorMessage()	Convierte errores de autenticación en mensajes amigables
Feedback en formularios	Mensajes de éxito/error con cierre manual

3. ErrorBoundary (captura de errores de renderizado)

- Componente de clase de React que envuelve la app y **detecta cualquier error lanzado durante el renderizado** de sus hijos.
- Cuando ocurre un error, muestra una pantalla de fallo con:
 - El mensaje *"Algo salió mal / Ocurrió un error inesperado. Podés intentar recargar la página."*
 - El **detalle técnico** del error (para diagnóstico).
 - Botones: **"Recargar página"** y **"Volver al inicio"**.
- Evita que la app quede en pantalla en blanco ante un error no controlado.

4. Friendly Firestore Error (friendlyFirestoreError)

Mapea los códigos de error de Firestore a mensajes accionables para el personal escolar:

Código	Mensaje mostrado
resource-exhausted	El servicio alcanzó su límite por hoy. Reintentá más tarde o avisá al supervisor.
unavailable	El servicio no responde en este momento. Reintentá en unos minutos.
network-request-failed	Problema de conexión. Verificá tu internet e intentá de nuevo.
permission-denied	No tenés permiso para realizar esta acción. Avisale al supervisor.
failed-precondition	La operación necesita una configuración pendiente del sistema. Avisale al supervisor.
cancelled	La operación fue cancelada. Intentá de nuevo.

Cualquier otro error cae en el mensaje genérico “Ocurrió un error inesperado. Intentá de nuevo.”

Nota: `resource-exhausted` (cuota agotada) se distingue de `network-request-failed`: la falta de cuota no queda en cola offline, mientras que la falta de red sí se sincroniza luego.

5. Friendly Auth Error (`getAuthErrorMessage`)

Mapea los códigos de Firebase Auth (login) a mensajes claros: usuario no existe, contraseña incorrecta, cuenta desactivada, demasiados intentos, sin conexión, etc. (detallado en el documento de Autenticación). Evita exponer detalles técnicos en el login.

6. Feedback en los formularios

- Cada formulario (asistencias, novedades, incidentes) muestra un mensaje de **feedback** al guardar:
 - **Éxito:** confirmación (y nota de sincronización si estuvo offline).
 - **Error:** usa `friendlyFirestoreError()` para mostrar un mensaje comprensible.
- Los mensajes tienen un **botón de cierre** `mid (x)` y además se auto-ocultan después de un tiempo.

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
<code>ErrorBoundary</code> global	Semana 3	~2-3 h
Mapeo de errores de autenticación (<code>getAuthErrorMessage</code>)	Semana 3	~1-2 h
Hook <code>useFeedback</code> (consolidar estados de operación)	Semana 3	~2 h
<code>friendlyFirestoreError</code> para errores de Firestore	Semana 5	~2 h
Feedback de éxito/error en formularios	Semanas 3-5	~3 h
Total aproximado	-	~1-1.5 días

8. Pendientes y observaciones

- Loggear errores a un servicio de monitoreo (Sentry) a futuro.
- Clasificar más códigos de error de Firestore si aparecen nuevos escenarios.

11 - Validación de Formularios

Este documento de la bitácora recopila el **sistema de validación** de los formularios, basado en **Zod** (schemas) integrado con **react-hook-form**.

1. Por qué Zod + react-hook-form

- **Zod**: define esquemas de validación **declarativos y tipados**. El mismo schema sirve para validar y para inferir el tipo TypeScript del formulario (`z.infer`).
- **react-hook-form**: maneja el estado de los campos y la integración con el DOM, con **renders mínimos** y buen rendimiento.
- `@hookform/resolvers/zod` → `zodResolver` : conecta ambos; la validación se ejecuta automáticamente con el schema.
- **Validación centralizada** (los mensajes y reglas viven en un solo lugar) y **tests** sobre los schemas.

2. Schemas de validación

2.1 Centralizados (`src/utils/validation.ts`)

`novedadSchema`

Campo	Regla
<code>fecha</code>	Obligatoria y no puede ser futura
<code>tipo</code>	Obligatorio (seleccionar un tipo)
<code>hora</code>	Opcional
<code>descripcion</code>	Mínimo 5 caracteres, máximo 500

`incidenteSchema`

Campo	Regla
<code>fecha</code>	Obligatoria y no futura
<code>categoria</code>	Obligatoria
<code>urgencia</code>	Obligatoria
<code>ubicacion</code>	Máximo 100 caracteres (opcional)
<code>descripcion</code>	Mínimo 10 caracteres, máximo 1000

Regla de fecha

- `fechaRule` valida que la fecha **no sea en el futuro**, usando `todayISO()` (fecha local del dispositivo, manejando la zona horaria).

2.2 Schemas locales (Supervisor)

Formulario de escuela (`SupervisorSchools.tsx`)

- `schoolSchema` : nombre/turno/etc. con valores obligatorios y mensajes de error.

Formulario de usuario (`SupervisorUsers.tsx`)

- `createUserSchema` : nombre (mín. 3), email (válido), contraseña (mín. 6), rol (enum), escuela (obligatoria).
- `editUserSchema` : igual pero sin contraseña (no se edita al modificar).

3. Cómo se integra

En cada formulario (Novedades, Incidentes, Usuarios, Escuelas):

```
const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting },
} = useForm<FormData>({
  resolver: zodResolver(schema),
  defaultValues: defaults,
});
```

- `register` conecta los campos.
- `Controller` se usa para componentes custom (ej. `DatePicker`).
- `errors.<campo>.message` se muestra junto a cada campo.
- Se puede combinar con `usewatch` para contadores (ej. longitud de descripción).

4. Validación adicional en la asistencia

El formulario de asistencia (`AttendanceForm`) tiene su **propia lógica de validación** (fuera de Zod, de forma manual):

- El **estado** de cada integrante es obligatorio.
- Si está **ausente**, el **motivo** es obligatorio.
- En modo secciones, si hay más de un integrante con el mismo cargo, el **nombre** es obligatorio.

5. Tests de validación

- Existen tests sobre los schemas (`src/test/validation.test.ts`) que verifican casos válidos e inválidos de `novedadSchema` e `incidenteSchema` (fechas futuras, descripciones cortas, campos faltantes, etc.).

6. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Schemas de novedades e incidentes (<code>validation.ts</code>)	Semana 1-2	~3 h
Integración con react-hook-form (<code>zodResolver</code>)	Semana 1-2	~3 h
Schema de escuela (migración a react-hook-form + Zod)	Semana 3	~2-3 h
Schemas de usuario (crear/editar)	Semana 3	~2-3 h
Validación manual de asistencia (motivo si ausente)	Semana 1-2	~2 h
Tests de validación	Semana 3	~2 h
Total aproximado	-	~1.5 días

7. Pendientes y observaciones

- Centralizar los schemas de usuario y escuela en `validation.ts` para consistencia.
- Considerar validación de tipos de imagen (ya se valida tamaño/tipo antes de subir fotos).

12 - Gestión de Estado y Hooks

Este documento de la bitácora recopila la **gestión de estado** y los **hooks** del proyecto: los providers/context de React, y la biblioteca de hooks reutilizables que componen la lógica de la interfaz.

1. Estrategia de estado

- **Estado global mínimo:** los estados compartidos que necesitan varios componentes se exponen por **React Context** (AuthContext, ToastContext). El resto es estado local de cada componente.
- **Hooks reutilizables:** la lógica repetitiva se extrae a hooks custom en `src/hooks/`, usados por varios componentes.
- **Persistencia puntual:** algunas preferencias (tema, borradores de formularios) se persisten en `localStorage`.

2. Providers / Contexts

AuthContext

- Maneja el estado de **autenticación**: `user`, `profile`, `isAuthenticated`, `isLoading`, y las funciones `login`, `logout`, `hasRole`, `canAccess`.
- Detallado en el documento de **Autenticación**.
- Incluye el **cierre de sesión por inactividad** del rol director.

ToastContext

- Sistema de **toasts** (avisos temporales): `addToast(type, message)`.
- Tipos: `success`, `error`, `warning`, `info`, con su icono y color.
- Cada toast se auto-elimina tras **4 segundos**, con **botón de cierre**.
- La lista de toasts se renderiza con `useAutoAnimate` (animación de entrada/salida) y se expone con `role="status"` y `aria-live="polite"` (accesibilidad).

3. Hooks reutilizables (`src/hooks/`)

Hook	Función
<code>useTheme</code>	Tema claro/oscuro + color primario (ver documento de UI)
<code>useOnlineStatus</code>	Detección de conexión (online/offline)
<code>useFeedback</code>	Estado de operación: <code>updatingId</code> , <code>feedback</code> , <code>start</code> , <code>end</code> , <code>clear</code> con auto-limpiado
<code>useFormDraft</code>	Auto-guardado de formularios en <code>localStorage</code> (restaura el borrador, con TTL)
<code>useHaptic</code>	Vibración del dispositivo (<code>navigator.vibrate</code>) con patrones <code>success/error/light</code>
<code>useSwipe</code>	Gestos de deslizar (touch) con umbral configurable
<code>useCountUp</code>	Contador animado hacia un valor objetivo (con easing)
<code>useKeyboardShortcuts</code>	Atajos de teclado (ej. <code>mod+k</code> , <code>escape</code>)
<code>useAmbientMotion</code>	Decide si las animaciones continuas corren (respetando <code>prefers-reduced-motion</code> , conexión, <code>saveData</code>)

`useFeedback`

- Centraliza los estados de “operación en curso / éxito / error” (`updatingId`, `feedback`).
- `start(id)` inicia la operación; `end(feedback)` la finaliza; el éxito se auto-limpia a los `FEEDBACK_AUTO_CLEAR_MS` segundos.
- Se usa en las pantallas de supervisión para mostrar progreso y resultados de guardado.

`useFormDraft`

- Guarda automáticamente los valores del formulario en `localStorage` (clave `signup-draft-{key}`).
- Restaura el borrador al volver, si no venció el **TTL** (24 h por defecto).
- Evita la pérdida de datos si el usuario se va sin enviar.

`useHaptic`

- Envuelve `navigator.vibrate` con **patrones por tipo** (success, error, light) y protege cuando no está soportado.

`useKeyboardShortcuts`

- Define atajos globales de teclado con un mapa { `tecla`: `handler` } (ej. `mod+k` abre búsqueda, `escape` cierra menús).

`useAmbientMotion`

- Controla las **animaciones ambientales** de fondo: las desactiva si hay `prefers-reduced-motion`, sin conexión, en datos móviles/ `saveData`, y las activa en Wi-Fi/Ethernet (o cuando la API no está disponible).

4. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
AuthContext (estado global de autenticación)	Semana 1	~4 h
ToastContext (sistema de toasts)	Semana 4	~3 h
useFeedback (estados de operación)	Semana 3	~2 h
useFormDraft (auto-guardado de formularios)	Semana 5	~3 h
useHaptic y useSwipe	Semana 5	~2 h
useCountUp (contadores animados)	Semana 4	~1-2 h
useKeyboardShortcuts	Semana 5	~2 h
useAmbientMotion (animaciones por conexión)	Semana 5	~2 h
useTheme y useOnlineStatus	Semanas 1-4	~2 h
Total aproximado	-	~2 días

5. Pendientes y observaciones

- Evaluar migrar a una librería de estado global (Zustand/Redux) si el estado crece.
- Considerar persistir el `updatingId` /feedback si se requiere más trazabilidad.

13 - Testing

Este documento de la bitácora recopila la **estrategia de testing** del proyecto y los **tests reales** ya implementados.

1. Stack de testing

Herramienta	Propósito
Vitest	Test runner (nativo de Vite, reemplaza Jest)
Testing Library (@testing-library/react)	Tests de componentes React
@testing-library/jest-dom	Matchers extra para aserciones de DOM
jsdom	Simulación del DOM en Node.js

2. Configuración

- En vite.config.ts: test.globals = true, test.environment = 'jsdom', setupFiles = './src/test/setup.ts'.
- src/test/setup.ts: import '@testing-library/jest-dom';
- Script: npm run test → vitest run.

3. Tests unitarios

Archivo	Qué cubre
src/test/validation.test.ts	Schemas Zod (novedadSchema , incidenteSchema): casos válidos e inválidos (fechas futuras, descripciones cortas, campos faltantes)
src/test/constants.test.ts	Constantes de la app (estados, roles, valores de negocio)
src/test/image.test.ts	Utilidades de imágenes (compresión/resize a base64)

4. Tests de componentes

Archivo	Qué cubre
Login.test.tsx	Formulario de inicio de sesión, validación y manejo de errores
Novedades.test.tsx / Incidentes.test.tsx	Formularios de registro y su validación
AttendanceRow.test.tsx	Fila de asistencia (estados, motivo si ausente)
IncidentHistory.test.tsx	Historial de incidentes y cambio de estado
StatusBadge.test.tsx	Renderizado según los 4 estados
RetentionBanner.test.tsx	Banner de retención/avisos
NotFound.test.tsx	Página 404

5. Smoke test global (all-components.smoke.test.tsx)

El test más importante: **monta TODAS las páginas y componentes comunes** para verificar que renderizan sin errores.

- **Stubs de APIs** que jsdom no implementa: `matchMedia`, `ResizeObserver`.
- **Mock de AuthContext** con rol configurable por test (director/supervisor), con valor estable para evitar bucles de efectos.
- **Mock de ToastContext** (`useToast`).
- **Mock de Firestore**: mockea las ~44 funciones de `firestore.ts` (queries y funciones `subscribe*` con `onSnapshot`) para datos seguros.
- Usa `createMemoryRouter` (Data Router) porque las páginas usan `<Link viewTransition>` / `useViewTransitionState`.
- Cubre: Login sin sesión, NotFound, Home (director/supervisor), Asistencia, Historial, Fotos, Novedades, Incidentes, Ayuda, Tema, y las páginas de Supervisor (Escuelas, Usuarios, Detalle). También ~18 componentes comunes (Button, DatePicker, StatusBadge, ErrorBoundary, Navbar, GlobalSearch, WelcomeTour, etc.).

Por qué importa: garantiza que un cambio no rompa el renderizado de ninguna pantalla ni componente, sin tocar Firestore real.

6. Cobertura mínima (referencia)

- Objetivo: Lines/Functions/Statements > 60%, Branches > 50%.
- Para el MVP la cobertura **no es bloqueante**; se prioriza testear la lógica crítica (validación, auth, servicios, formularios) y el smoke test global.

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Configuración de Vitest + setup	Semana 3	~1-2 h
Tests unitarios (validación, constantes, imagen)	Semana 3	~3 h
Tests de componentes (Login, Novedades, Incidentes, etc.)	Semanas 3-5	~5 h
Smoke test global (mocks + montaje de todas las pantallas)	Semanas 4-5	~6 h
Total aproximado	-	~1.5-2 días

8. Pendientes y observaciones

- Agregar tests de integración de servicios (`firestore.ts` con emulador de Firestore).
- Agregar tests más específicos de formularios (submit, validación en pantalla).
- Considerar generación de reporte de cobertura para visualizar avance.

14 - Rendimiento

Este documento de la bitácora recopila las **optimizaciones de rendimiento** aplicadas al proyecto (una PWA enfocada a celulares y redes de baja calidad en escuelas rurales de Tinogasta).

1. Contexto

- La app recorre **celulares con conexiones limitadas y a veces sin internet**.
- Las **fotos** (asistencia, partes) se guardan como **base64 en Firestore**, que limita cada documento a **1 MB**. Comprimir bien las imágenes es crítico.
- Ser **offline-first** (con Workbox/Service Worker) es la principal mejora percibida de rendimiento.

2. Optimizaciones aplicadas

2.1 Compresión de imágenes (src/utils/image.ts)

- `fileToCompressedDataUrl(file)` : redimensiona la imagen en un `<canvas>` a **máx 1024px** en su lado mayor y la codifica como **JPEG con calidad 0.6**.
- Límites de seguridad:
 - Entrada: `MAX_IMAGE_INPUT_BYTES` = **20 MB** (físico de celulares).
 - Salida: `MAX_SAFE_DATA_URL_LENGTH` = **900 KB** (por debajo del límite de 1 MB de Firestore, dejando margen para el resto del documento).
- `validateImageFile` rechaza archivos que no sean imagen o superen 20 MB con mensaje claro.
- Evita que surja el error de "documento demasiado grande" y reduce los datos subidos por la red.

2.2 Service Worker / PWA (vite.config.ts)

- `vite-plugin-pwa` con `registerType: 'autoUpdate'` (la app se actualiza sola en segundo plano).
- **Precache** de `js, css, html, svg, png, woff2` : la app carga offline al instante.
- **runtimeCaching con NetworkFirst** para las llamadas a `firestore.googleapis.com` : intenta la red y, si no hay conexión o falla, sirve desde la caché.
- Manifiesto con `display: 'standalone'`, íconos y **shortcuts** a pantallas frecuentes (Asistencia, Historial, Ayuda).

2.3 Animaciones eficientes

- `useCountUp` : anima contadores con `requestAnimationFrame` + easing cúbico (rápido y sin bloquear el hilo).
- `useAmbientMotion` : **desactiva las animaciones ambientales** cuando hay conexión móvil / `saveData` / sin conexión / `prefers-reduced-motion` ; las deja solo en Wi-Fi/Ethernet. Ahorra batería y datos.
- Las animaciones de la UI usan transformadas CSS (GPU) en lugar de propiedades que fuerzan layout.

2.4 Renders mínimos

- **Lazy loading por ruta**: el enrutador (`src/routes/index.tsx`) usa `lazy: () => load(() => import(...))` CON `<Suspense fallback={}<LoadingScreen /></Suspense>` — cada pantalla carga su propio chunk solo al visitarla.

- **react-hook-form** minimiza los re-renders de formularios (el estado de cada campo se registra sin re-renderizar todo el árbol).
- Estado local en lugar de re-render global (solo los contexts necesarios comparten estado).

3. Observaciones (pendientes)

- Hay **lazy loading por ruta**: el enrutador (`src/routes/index.tsx`) usa `lazy: () => load(() => import(...))` envuelto en un `<Suspense fallback={ <LoadingScreen /> }>` en `main.tsx` . Cada pantalla carga su propio chunk de JS solo cuando se visita.
- **No hay `React.memo` / `useMemo`** en el código; aunque el bundle ya está dividido por rutas, **se puede mejorar** memoizando listas/derivaciones costosas donde aporte.
- Evaluar medición de rendimiento real (Lighthouse / Web Vitals) en [deploy].

4. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Compresión de imágenes (canvas → JPEG base64)	Semana 5	~4 h
Configuración PWA (manifiesto + workbox)	Semanas 4-5	~4 h
Cache <code>NetworkFirst</code> para Firestore	Semana 5	~2 h
Animaciones con <code>requestAnimationFrame</code>	Semana 4-5	~2 h
<code>useAmbientMotion</code> (desactiva según conexión)	Semana 5	~2 h
Total aproximado	-	~1.5 días

5. Pendientes / próximos pasos

- Implementar **lazy loading** de rutas y memoización donde aporte.
- Medir Core Web Vitals y tamaño de bundle (`vite-bundle-visualizer`).

15 - PWA y Móvil

Este documento de la bitácora recopila cómo la app se comporta como una **PWA (Progressive Web App)** enfocada a **uso móvil**, pensada para celulares del personal escolar en Tinogasta.

1. Por qué PWA

- Los directores y docentes usan la app **desde el celular**, a menudo con **conexión intermitente**.
- Una PWA permite **instalarla** en la pantalla de inicio (como una app nativa, sin pasar por tiendas) y **funcionar offline**.
- La app es **mobile-first**: diseñada y probada primero para pantallas chicas y táctiles.

2. Configuración PWA (vite.config.ts)

Manifiesto

- `name`: SIPNAM — Sistema Integrado de Partes de Novedades y Asistencias Móvil.
- `display`: 'standalone', `orientation`: 'portrait', `theme_color`: #1e40af.
- **Íconos** 192x192 y 512x512 (incluido `maskable`).
- **Shortcuts** para acciones rápidas al mantener presionado el ícono:
 - Cargar asistencia (/asistencia)
 - Ver historial (/historial)
 - Centro de ayuda (/ayuda)

Service Worker (Workbox)

- `registerType`: 'autoUpdate': la app se **actualiza sola** en segundo plano.
- **Precache** de `js`, `css`, `html`, `svg`, `png`, `woff2`.
- **NetworkFirst** para Firestore: offline-tolerante (ver documento de Tiempo Real y Offline).

3. Meta tags móviles (index.html)

- `viewport-fit=cover` (para aprovechar pantallas con notch).
- `theme-color` acorde al color primario.
- `mobile-web-app-capable` / `apple-mobile-web-app-*` → soporte iOS.
- `apple-touch-icon` para el ícono en iOS.

4. Instalación (InstallPrompt)

- Escucha el evento `beforeinstallprompt` del navegador y lo muestra como **invitación a instalar**.
- **Detección de standalone**: no muestra el aviso si la app ya está instalada o en modo standalone.
- **Modo iOS**: como iOS no dispara `beforeinstallprompt`, muestra instrucciones manuales ("Compartir → Agregar a pantalla de inicio").
- El aviso aparece tras un **delay** (`SHOW_DELAY_MS` ~2.5s) y se puede **descartar** (persistido en `localStorage` `sipnam-install-dismissed`).

- Se oculta automáticamente cuando se dispara el evento `appinstalled`.

5. UX móvil / táctil

- `BottomNav` : barra de navegación inferior (patrón de app móvil) con menú lateral ("drawer") para más opciones.
- `useSwipe` : gestos de deslizar con el dedo (con umbral configurable).
- `useHaptic` : vibración háptica en acciones (exitosa/error).
- `useKeyboardShortcuts` : atajos para usuarios de escritorio.
- `useCountUp` y animaciones suaves adaptadas al tacto.
- **Controles táctiles grandes** y mensajes claros para dedos (ver documento de UI).
- **Análisis de conectividad:** `useOnlineStatus` y `ConnectionBanner` avisan cuándo hay/sin conexión.

6. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Configuración PWA (manifiesto + workbox)	Semanas 4-5	~4 h
Service Worker con <code>NetworkFirst</code>	Semana 5	~2 h
<code>InstallPrompt</code> (invitación a instalar + modo iOS)	Semana 5	~3 h
Meta tags móviles en <code>index.html</code>	Semana 1-4	~1 h
<code>BottomNav</code> y navegación móvil	Semanas 3-5	~3 h
Gestos táctiles (<code>useSwipe</code> , <code>useHaptic</code>)	Semana 5	~2 h
Total aproximado	-	~1.5 días

7. Pendientes y observaciones

- Verificar el ícono `maskable` con el área segura.
- Probar instalación en iOS y Android reales.
- Considerar notificaciones `push` locales al caché (ver documento de Notificaciones).

16 - Accesibilidad

Este documento de la bitácora recopila las **prácticas de accesibilidad** implementadas en la app, orientadas a **WCAG 2.1 nivel AA**.

1. Estándar y objetivo

- Se busca cumplir **WCAG 2.1 nivel AA**, enfocado en los puntos más relevantes para una app web administrativa que también se usa en celular.
- La accesibilidad mejora la usabilidad para **todos**: personas con discapacidad visual, motriz o cognitiva, y también en contextos con pantallas chicas o luz solar.

2. ARIA en el código

Uso efectivo de atributos ARIA en los componentes reales:

Atributo	Cantidad	Uso
<code>aria-label</code>	22	Botones de icono sin texto visible (descartar, cerrar, acciones)
<code>aria-hidden</code>	6	Iconos decorativos
<code>aria-modal</code>	4	Diálogos/modales (ConfirmDialog, GlobalSearch, Lightbox, WelcomeTour)
<code>aria-live</code>	3	Regiones dinámicas (toasts con <code>aria-live="polite"</code>)
<code>aria-current</code>	1	Elemento activo de navegación

- `role="status" + aria-live="polite"` en el contenedor de toasts (`ToastContext`): los lectores de pantalla anuncian los avisos sin interrumpir.
- `role="dialog" + aria-modal="true"` en los modales, con `aria-labelledby` apuntando al título.
- `role="alert"` para mensajes de error críticos en los formularios.

3. Semántica HTML y landmarks

- Uso de elementos semánticos: `<header>`, `<nav>`, `<main>`, `<footer>`, `<section>`, `<form>`, `<table>`, y jerarquía de títulos `<h1>` – `<h6>` con un solo `<h1>` por página.
- Formularios con `<label>` asociado a cada campo (`id / for`).
- Errores ligados al campo con `aria-describedby` y `aria-invalid`.

4. Navegación por teclado

- **Focus visible**: estilo de foco (`:focus-visible`) con `outline` de color primario.
- **Atajos de teclado** (`useKeyboardShortcuts`): soporta `mod`, `shift`, `alt` combinados; incluye acciones como cerrar con `Escape`.
- Componentes interactivos con `onKeyDown` y `tabIndex` donde hace falta (GlobalSearch, AttendanceForm).
- Las utilidades (`useSwipe`) **no sustituyen** a la interacción por teclado (todo lo crucial es accesible por teclado además del gesto táctil).

5. Movimiento reducido (`prefers-reduced-motion`)

- `useReducedMotion` (de `motion/react`) se usa en: `ConfirmDialog`, `RetentionBanner`, `Lightbox` y `useAmbientMotion`; respeta la preferencia del usuario de reducir movimiento.
 - `useAmbientMotion` **desactiva las animaciones ambientales continuas** cuando hay `prefers-reduced-motion`, además de cuando no hay conexión o se usa datos móviles/ `saveData` (accesibilidad + ahorro de batería).
-

6. Contraste y color

Paleta verificada contra WCAG AA sobre fondo claro:

Elemento	Contraste	Estado
Texto principal (<code>#1e293b</code> / <code>#f8fafc</code>)	14.5:1	☐
Texto secundario (<code>#64748b</code>)	5.0:1	☐
Botón primario (texto blanco / <code>#1e40af</code>)	8.6:1	☐
Botón peligro (blanco / <code>#dc2626</code>)	4.6:1	☐
Enlace (<code>#2563eb</code>)	7.1:1	☐

- El modo **dark** también se define con variables de color con contraste verificado.
 - Los estados de los registros (pendiente, resuelto, etc.) usan **texto + icono**, no solo color, para no depender únicamente del color.
-

7. Imágenes e iconos

- Imágenes informativas con `alt` descriptivo.
 - Iconos decorativos con `aria-hidden="true"`.
-

8. Testing de accesibilidad

- `@axe-core/react` está instalado (paquete en `package.json`).
 - Auditoría manual: navegación solo con teclado, uso de lector de pantalla y verificación de contraste (WCAG AA).
-

9. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
ARIA (labels, roles, modales, live regions)	Semanas 3-5	~4 h
Semántica HTML y labels de formularios	Semanas 1-3	~3 h
<code>prefers-reduced-motion</code> / <code>useReducedMotion</code>	Semana 5	~2 h
Contraste y paleta verificada	Semanas 1-4	~2 h
Integración de <code>@axe-core/react</code>	Semanas 4-5	~1 h
Total aproximado	-	~1-1.5 días

10. Pendientes y observaciones

- Completar la inicialización de `@axe-core/react` en modo desarrollo (si aún no está activa).
- Añadir tests de accesibilidad con `jest-axe` o auditoría automatizada en CI.
- Revisar `aria-invalid` / `aria-describedby` en todos los formularios editables.

17 - UX y Onboarding

Este documento de la bitácora recopila las **estrategias de UX (experiencia de usuario)** y de **onboarding** para que el personal escolar aprenda a usar la app rápidamente y sin frustración.

1. Principios de UX

- **Los roles definen la experiencia:** cada usuario solo ve y hace lo que le corresponde (según su rol), reduciendo la complejidad percibida.
- **Atajos y flujos rápidos** para las tareas diarias (asistencia, foto de planilla, partes).
- **Estados vacíos atractivos:** cuando no hay datos, se muestra una vista guía en lugar de una pantalla “en blanco”.
- **Guiones útiles en contexto** y un **centro de ayuda** completo con preguntas frecuentes.

2. Onboarding: WelcomeTour

- **Tour de bienvenida** que aparece la **primera vez** que cada usuario inicia sesión.
- **Personalizado según el rol:**
 - **Director/Vice:** explica Asistencia, Novedades/Incidentes y Seguimiento.
 - **Preceptor:** explica Asistencia y la Foto diaria de la planilla.
 - **Supervisor:** explica el Panel de Supervisión, la Verificación/Seguimiento y la Administración (usuarios, escuelas, exportación).
- Experiencia paso a paso con **contador** (“Paso X de Y”), puntitos de progreso y navegador “Saltar”/“Siguiente”/“Empezar”.
- **Accesible:** `role="dialog"`, `aria-modal`, se cierra con `Escape`, bloquea el scroll de fondo.
- **Persistencia:** se marca como visto en `localStorage` (clave con el `uid` del usuario) para no repetirse en cada visita.

3. Pistas contextuales: ContextHint

- **Notas de ayuda desplegables** junto a secciones clave de la app.
- Se pueden **descartar** con un botón (x), y el descarte se **guarda** (`signup-hint-dismissed-{id}`) para no volver a mostrarse.
- `role="note"` para lectores de pantalla.
- Ayudan a orientar sin interrumpir el flujo principal.

4. Estados vacíos: EmptyState

- Cuando una sección no tiene registros, se muestra un **estado vacío** con:
 - Icono representativo (asistencia, novedades, alerta, usuarios, etc.).
 - Título y descripción breve.
 - **Acción sugerida** (botón “Cargar...” que redirige a la pantalla correspondiente o ejecuta una acción).
- Convierte el momento “no hay nada” en una **invitación a comenzar**.

5. Centro de ayuda: Ayuda

- Página de ayuda con **FAQ filtrado por rol** (las preguntas que no aplican al rol del usuario se ocultan).
- Docenas de respuestas concisas: escuela automática, límites de carga por rol, permisos de eliminación de fotos, etc.
- **Secciones** adicionales: **modo offline** (`wifiOff`), **cómo instalar la app** (`Smartphone`) y **glosario** (`BookOpen`).
- Accesible desde el **tour de bienvenida** ("Saltar" lo lleva directamente a Ayuda) y desde la navegación.

6. Otras mejoras de UX

- **Feedback inmediato y claro** en cada acción (éxito/error, toasts) — ver Manejo de Errores.
- **Auto-guardado de borradores** (`useFormDraft`) para no perder datos en formularios incompletos — ver Estado y Hooks.
- **Vibración háptica** (`useHaptic`) en acciones para feedback táctil.
- **Animaciones suaves** (ver documento de UI) con respeto a `prefers-reduced-motion`.

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
WelcomeTour (pasos + persistencia por rol)	Semana 5	~6 h
ContextHint (pistas contextuales)	Semana 5	~2 h
EmptyState (estados vacíos guiados)	Semanas 3-5	~2 h
Página Ayuda (FAQ por rol + secciones)	Semanas 4-5	~4 h
Micro-interacciones (haptic, toasts, borradores)	Semanas 4-5	~3 h
Total aproximado	-	~1.5-2 días

8. Pendientes y observaciones

- Ampliar el tour con capturas o pasos interactivos si se detectan fricciones reales.
- Analizar con usuarios reales (pruebas de usabilidad) qué flujos generan más dudas.

18 - Seguridad

Este documento de la bitácora recopila las **medidas de seguridad** del proyecto: autenticación, permisos por rol, reglas de Firestore y protección del acceso.

1. Capas de seguridad

Capa	Descripción
Autenticación	Firebase Auth (email/contraseña), sesión gestionada en <code>AuthContext</code>
Autorización	Permisos por rol y por escuela (<code>hasRole</code> , <code>canAccess</code>)
Reglas de Firestore	Reglas de backend que validan cada lectura/escritura
Cierre de sesión por inactividad	Protege el acceso en dispositivos compartidos
Validación de entrada	Zod + validación de imágenes (límites de tamaño)

2. Autenticación y sesión

- Login con **email/contraseña** (se verifica el perfil en la colección `usuarios`).
- El `AuthContext` mantiene el estado de sesión y expone `user` , `profile` , `isAuthenticated` .
- **Cierre de sesión por inactividad** para el rol **director** (10 min) — protege equipos compartidos en las escuelas. El **supervisor** no tiene timeout.
- Los errores de autenticación se traducen a mensajes amigables sin exponer detalles técnicos (ver Manejo de Errores).

3. Autorización por rol

- Cada pantalla comprueba el rol del usuario antes de mostrar información sensible (`hasRole` , `canAccess` en `AuthContext`).
- El **supervisor** tiene acceso total; los roles de escuela (director, vice, preceptor) solo ven y operan sobre **su propia** `escuelaId` .
- `secretario` y `conserje` solo quedan registrados en la asistencia (no acceden a la app).

4. Reglas de Firestore (firestore.rules)

Las reglas se aplican en el **backend** (no confían solo en la UI):

Funciones auxiliares

- `isSignedIn / hasProfile` : exige sesión y perfil en `usuarios/{uid}` .
- `isSupervisor()` : rol `supervisor` .
- `userBelongsToSchool(schoolId)` : el usuario pertenece a esa escuela.
- `canSeeSchool(schoolId)` : supervisor o pertenece a la escuela.
- `onlyChangedFields(fields)` : restringe qué campos puede modificar una actualización.

Resumen por colección

Colección	Crear	Leer	Actualizar	Eliminar
escuelas	Supervisor	Ver por escuela	Supervisor	Supervis
usuarios	Supervisor	Ver según rol	Propio (solo nombre/email) o Supervisor	Supervis
asistencias	director/vice/preceptor de su escuela	Ver por escuela	Supervisor (solo verificada ...)	□
asistencia_docentes	director/vice/preceptor de su escuela	Ver por escuela	Supervisor (solo verificada ...)	□
fotos	preceptor de su escuela	Ver por escuela	□	Supervis o propietaria de la foto
novedades	director/vice	Ver por escuela	□	□
incidentes	director/vice	Ver por escuela	Supervisor (solo estado / updatedAt)	□
docentes	Supervisor	Ver por escuela	Supervisor	Supervis

Principios clave: - No-supervisores solo leen su propia escuela

(`canSeeSchoolData`). - En creaciones, se fuerza que `cargadoPor == request.auth.uid` y que la escuela coincida con la del usuario (evita suplantación). - Las **actualizaciones** solo permiten cambiar campos específicos (`onlyChangedFields`) — ej. el supervisor no puede reescribir toda la asistencia, solo marcarla verificada; el autor no puede editar incidentes/novedades. - **Eliminaciones** de registros de asistencia/incidentes/novedades están **prohibidas** (auditoría); las fotos solo las borra el autor o el supervisor.

5. Validación de entrada

- **Zod** valida cada formulario (mensajes claros y reglas tipadas) — ver Validación de Formularios.
- **Imágenes** (`src/utils/image.ts`): se validan tipo y tamaño (máx 20 MB) **antes** de procesar, y se comprimen a ≤ 900 KB (data URL) para no exceder el límite de 1 MB de Firestore; evita abusos de tamaño.

6. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Autenticación y sesión en <code>AuthContext</code>	Semana 1	~4 h
Permisos por rol (<code>hasRole</code> / <code>canAccess</code>)	Semanas 1-2	~2 h
Cierre de sesión por inactividad (director)	Semana 3	~2 h
Reglas de Firestore por colección	Semana 2	~6 h
Validación de imágenes (tipo/tamaño)	Semana 5	~2 h
Total aproximado	-	~1.5-2 días

7. Pendientes y observaciones

- Verificar las reglas con el **emulador de Firestore** ante casos límite (campos extra, IDs).
- Considerar **verificación de email** al crear usuarios.
- Revisar que `usuario.email` no sea editable en demasía y evaluar `firestore.indexes.json` para consultas.

19 - Despliegue

Este documento de la bitácora recopila la **estrategia de despliegue** de la app (una PWA en React/Vite) y la configuración real utilizada en el repositorio.

1. Visión general

- La app es una **SPA estática** (React + Vite): el build genera archivos estáticos en `dist/` que se sirven desde un hosting.
- Las **reglas e índices de Firestore** se despliegan por separado con la CLI de Firebase.
- Se usa **Netlify** como hosting, con **deploy continuo desde Git**.

2. Build

- Script: `npm run build` → `tsc -b && vite build`.
- Genera la carpeta `dist/` lista para servir.
- El build incluye el **Service Worker (PWA)** generado por `vite-plugin-pwa`.

3. Configuración efectiva: Netlify (`netlify.toml`)

```
[build]
  command = "npm run build"
  publish = "dist"
```

Redirects (SPA)

- Se redirige `/*` → `/index.html` con status 200, de modo que el **routing del lado del cliente** (React Router) funcione al recargar cualquier ruta (p. ej. `/historial`).

Headers / caché

- `/assets/*`, `*.js` y `*.css`: `Cache-Control: public, max-age=31536000, immutable` (cacheo de largo plazo de archivos con hash).
- `/index.html`: `Cache-Control: no-cache` (siempre se revalida para obtener la última versión de la app).

4. Configuración de Firebase (`firebase.json`)

- Solo se despliegan las **reglas** (`firestore.rules`) y los **índices** (`firestore.indexes.json`) de Firestore.
- Deploy: `firebase deploy --only firestore:rules` (y `:indexes` cuando cambian).

5. Variables de entorno

- Las credenciales de Firebase se leen de variables `import.meta.env.VITE_*` en `src/services/firebase.ts`.

- Se definen en el archivo `.env` local y como **variables de entorno en Netlify** (producción) (no se suben al repo).

Variable	Valor
VITE_FIREBASE_API_KEY	API key
VITE_FIREBASE_AUTH_DOMAIN	Auth domain
VITE_FIREBASE_PROJECT_ID	Project ID
VITE_FIREBASE_MESSAGING_SENDER_ID	Sender ID
VITE_FIREBASE_APP_ID	App ID

`.env.example` documenta las variables requeridas; `.env` no se versiona.

6. Flujo de deploy

1. `npm run build` (genera `dist/`).
2. Netlify publica `dist/` con los redirects y headers configurados.
3. Si cambian las reglas/índices de Firestore, se despliegan con la CLI de Firebase.
4. Verificación post-deploy: HTTPS activo, login de prueba, sincronización online/offline y comportamiento en móvil.

Nota: la documentación original de despliegue recomendaba **Firebase Hosting**, pero la **implementación efectiva** en el repo usa **Netlify** (con `netlify.toml`). Ambos son válidos para una SPA estática.

7. Tiempo estimado por tarea

Tarea	Cuándo	Tiempo estimado
Configurar Netlify (<code>netlify.toml</code>): build, publish, redirects	Semana 5	~2 h
Headers de caché (assets inmutables, index no-cache)	Semana 5	~1 h
Configurar variables de entorno de Firebase	Semana 5	~1 h
Despliegue de reglas/índices de Firestore	Semana 2	~1 h
Verificación post-deploy (HTTPS, login, offline, móvil)	Semana 5	~2 h
Total aproximado	-	~0.5-1 día

8. Pendientes y observaciones

- Configurar **deploy continuo automático** desde la rama `main` de Git (Netlify lo soporta).
- Considerar rama de **preview** por PR antes de producción.
- Verificar el **Service Worker** (auto-update) en el dominio final de Netlify.

Semana N — Título de la semana

Período: días X al Y de MES de 2026

Horas acumuladas: [completar]

Objetivo de la semana

[Qué se buscó lograr esta semana en una o dos líneas.]

Actividades realizadas

Día 1 de la semana

- [Descripción de la actividad con detalle técnico, ej.: "implementé el cierre de sesión por inactividad para el rol director en AuthContext.tsx"]
 - [Sub-detalle o evidencia]

Día 2 de la semana

- [Actividad]

Consejo: anotá lo que hiciste el mismo día en borrador y pulí la semana al cierre. Mencioná evidencias (commits, pantallas, "pruebas pasaron").

Dificultades encontradas

- [Problema y cómo se resolvió]
 - [Problema y cómo se resolvió]
-

Resultados y evidencias

- [Qué quedó funcionando]
 - [N.º de commits y su alcance]
-

Aprendizajes

- [Tecnología/herramienta y qué aprendiste]
-

Pendientes / Próxima semana

- [Qué sigue]